HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

BACHELOR THESIS

Topic:

ADAPTIVE PARTITIONING METHODS FOR 3D TILES

Student: VU TONG MINH HIEU
Student ID: 20193218
Class: CTTT DT01 – K64
Instructor: ASSOC. PROF. DAO TRUNG KIEN

Hanoi, 8-2023

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING

BACHELOR THESIS

Topic:

ADAPTIVE PARTITIONING METHODS FOR 3D TILES

Student: VU TONG MINH HIEU

Student ID: 20193218

Class: CTTT DT01 – K64

Instructor: ASSOC. PROF. DAO TRUNG KIEN

Reviewer: .....

Hanoi, 8-2023

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

(DÀNH CHO CÁN BỘ HƯỚNG DẪN)

Tên đề tài: Adaptive Partitioning Methods For 3D Tiles.

Họ tên sinh viên: Vũ Tống Minh Hiếu MSSV: 20193218

Cán bộ hướng dẫn: PGS. TS. Đào Trung Kiên

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	**Thái độ làm việc (2,5 điểm)**	Nghiêm túc, tích cực và chủ động trong quá trình làm ĐATN	2,5
		Hoàn thành đầy đủ và đúng tiến độ các nội dung được GVHD giao	
2	**Kỹ năng viết quyển ĐATN (2 điểm)**	Trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v.	2,0
		Kỹ năng diễn đạt, phân tích, giải thích, lập luận: Cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
3	**Nội dung và kết quả đạt được (5 điểm)**	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận.	5,0
		Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan.	
		Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	**Điểm thành tích (1 điểm)**	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. **(1 điểm)**	0,0
		Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. **(0,5 điểm)**	
		Điểm tổng các tiêu chí:	9,5
		Điểm hướng dẫn:	9,5

Cán bộ hướng dẫn
(Ký và ghi rõ họ tên)

Đào Trung Kiên

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP
(DÀNH CHO CÁN BỘ PHẢN BIỆN)

Tên đề tài: Adaptive Partitioning Methods For 3D Tiles.

Họ tên sinh viên : Vũ Tống Minh Hiếu MSSV: 20193218

Cán bộ hướng dẫn : PGS. TS. Đào Trung Kiên

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	**Trình bày quyển ĐATN (4 điểm)**	Đồ án trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v.	
		Kỹ năng diễn đạt, phân tích, giải thích, lập luận: cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
2	**Nội dung và kết quả đạt được (5,5 điểm)**	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận.	
		Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan.	
		Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/ tính sáng tạo trong nội dung và kết quả đồ án.	
3	**Điểm thành tích (1 điểm)**	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. **(1 điểm)**	
		Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. **(0,5 điểm)**	
		Điểm tổng các tiêu chí:	
		Điểm phản biện:	

Cán bộ phản biện
(Ký và ghi rõ họ tên)

PREFACE

This thesis, titled “Adaptive Partitioning Methods for 3D Tiles”, is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Electronics and Telecommunication at School of Electrical and Electronics, Hanoi University of Science and Technology. The research work presented herein has been conducted under the guidance and supervision of Assoc. Prof. Dao Trung Kien, who has provided invaluable support and mentorship throughout the entire journey.

The ever-increasing demand for three-dimensional (3D) data in various fields, including virtual reality, computer-aided design, and geospatial applications, has led to the development of efficient data structures for managing and transmitting large-scale 3D datasets. One such data structure is the concept of 3D tiles, which provides a means of organizing, streaming, and rendering geospatial data with different levels of detail.

The main objective of this thesis is to investigate and propose partitioning methods for enhancing the performance and efficiency of 3D tiles. Traditional partitioning techniques often rely on fixed partitioning schemes, which may lead to suboptimal performance when dealing with complex and dynamic 3D datasets. In contrast, adaptive partitioning methods aim to dynamically adjust the partitioning scheme based on the characteristics of the data and the rendering context.

Throughout this thesis, we delve into the fundamental principles of 3D tiles and explore existing partitioning methods. We critically analyze their limitations and propose novel adaptive partitioning algorithms to overcome these challenges. Our research focuses on improving the efficiency of rendering large-scale 3D datasets by dynamically adjusting the partitioning scheme based on the user's viewpoint, the level of detail required, and other contextual factors.

I would like to express my deepest gratitude to Assoc. Prof. Dao Trung Kien for his constant guidance, insightful suggestions, and continuous support throughout the research process. His expertise and commitment have been instrumental in shaping the direction and quality of this thesis.

I would also like to thank my colleagues, friends, and family members for their encouragement and assistance throughout this endeavor. Their unwavering support has been a constant source of inspiration.

DECLARATION OF AUTHORSHIP

I am Vu Tong Minh Hieu, student ID 20193218, Advanced Program in Electronics and Telecommunication K64. My instructor is Assoc. Prof. Dao Trung Kien. I assure that all the content presented in the thesis *Adaptive Partitioning Methods for 3D Tiles* is the result of my research. The data stated in the project is completely honest, reflecting the actual measurement results. All cited information complies with intellectual property regulations. The references are clearly listed. I take full responsibility for the content written in this project.

Hanoi, August 10, 2023

Author

Vu Tong Minh Hieu

TABLE OF CONTENTS

NOMENCLATURE ..... **i**
LIST OF FIGURES ..... **ii**
LIST OF TABLES ..... **iv**
ABSTRACT ..... **v**
INTRODUCTION ..... **vi**
CHAPTER 1. THEORETICAL BASIS **1**
1.1 3D Tiles Specifications 1
1.1.1. Tileset 3
1.1.2. Tileset and Tiles 4
1.1.3. Bounding Volume 6
1.1.4. Geometric Error 7
1.1.5. Refinement 8
1.1.6. Transform in 3D Tiles 9
1.1.7. Tile Format Specifications 10
1.2. GL Transmission Format 10
1.2.1. Overview 11
1.2.2. Coordinate System and Units 11
1.2.3. glTF Object Hierarchy 11
1.2.4. Scenes 12
1.2.5. Nodes and Hierarchy 12
1.2.6. Buffers and Buffer Views 13
1.2.7. Accessors 14
1.2.7.1 Overview 14
1.2.7.2 Accessor Data Types 15
1.2.7.3 Accessors Bounds 16
1.2.8. Meshes 16
1.2.9. Getting Data of Attributions 17
1.2.10. Texture and Material Data 17
1.3. glTF Binary File Format Specification 18
1.3.1. Overview 18
1.3.2. Structure 19
1.3.3. Layout 19

1.3.4. glTF to GLB Transformation 20
1.4. Batched 3D Model 20
1.4.1. Overview 20
1.4.2. Layout 21
1.4.3. Convert GLB to B3DM 22
1.5. Sample Tiling Methods 22
1.5.1. Quadtrees Tiling 22
1.5.2. K-d Trees Tiling 25
1.5.3. Octrees Tiling 26
CHAPTER 2. PROPOSED PARTITIONING METHODS **27**
2.1. Overview 27
2.1.1. Objectives 27
2.1.2. Specifications 27
2.1.3. Partitioning Process Block Diagram 27
2.1.4. Sample Application Diagrams 28
2.2. Proposed Methods 29
2.2.1. Static Octree Tiling 30
2.2.2. Adaptive Octree Tiling 30
2.3. Making Dataset For Tiling 31
2.3.1. OpenStreetMap 31
2.3.2. OSM2World 32
2.4. Determining if a Triangle is in a Tile 33
CHAPTER 3. ALGORITHM DEVELOPMENT **34**
3.1. Data Extraction 34
3.2.1. Parsing glTF 35
3.2.2. Accessing Vertex Information 35
3.2.2.1 Get Accessors 35
3.2.2.2 Get Buffers 37
3.2.2.3 Get ByteStrides 38
3.2.2.4 Get Offsets 39
3.2.2.5 Read Data 39
3.2.2.6 Read & Update Each Attribute List 40
3.2.2.7 Calculate Root Bounding Volume 43
3.2. Tile Partitioning 44

3.3.1 Declare Tile 45
3.3.2 Declare List of Tiles 46
3.3.3 Static Octree Tiling 46
3.3.4 Adaptive Octree Tiling 46
3.3.5 Repeated Partitioning Process 48
3.3.5.1 Divide into 8 Tiles (Octree) 48
3.3.5.2 Checking conditions and further subdivisions 50
3.3. Tileset Complication 51
3.4.1. Updating Full Information of Each Tile 52
3.4.1.1 Updating boundingVolume of Each Tile 53
3.4.1.2 Updating indiceList, positionList, normalList, colorList of Each Tile 53
3.4.2. Converting a Tile to B3DM Format 55
3.4.2.1 Converting a Tile to glTF Format 55
3.4.2.2 Converting a glTF File to GLB Format 57
3.4.2.3 Converting a GLB File to B3DM Format 58
3.4.3. Write JSON Tileset 58
CHAPTER 4. IMPLEMENTATION AND EVALUATION 61
4.1. Implementation on CesiumJS 61
4.2. Demo Application 62
4.2.1 Application Explanation 62
4.2.2 Result 63
4.3. Measurement 64
4.3.1. Same Maximum Number Of Triangles Per Tile 64
4.3.1.1 2000 Triangles Per Tile 65
4.3.1.2 3000 Triangles Per Tile 66
4.3.2. Measurement Conclusions 68
CONCLUSION 69
General conclusion 69
Future works 69
REFERENCES 70
APPENDIX 71

NOMENCLATURE

Abbreviation	Meanings
glTF	GL Transmission Format
GLB (glb)	glTF Binary File Format
B3DM (b3dm)	Batched 3D Model
HLOD	Hierarchical Levels of Detail
LOD	Levels Of Detail
WebGL	Web Graphics Library
2D	2 Dimensions
3D	3 Dimensions
JSON (json)	JavaScript Object Notation
URI (uri)	Uniform Resource Identifier
CAD	Computer-Aided Design
WGS 84	World Geodetic System 1984
EPSG 4979	Geodetic parameter dataset
SSE	Screen-space Error
GPU	Graphics Processing Unit
K-d tree	K-dimensional tree
OSM (osm)	OpenStreetMap

LIST OF FIGURES

Figure 1.1 Sample of an application using 3D Tiles [2] 1
Figure 1.2 Hierarchical structure of a Tileset [4] 4
Figure 1.3 Sphere Bounding example [4] 6
Figure 1.4 Box Bounding example [4] 6
Figure 1.5 Region Bounding example [4] 7
Figure 1.6 Example of how geometricError effect the visible level on the screen [4] ... 7
Figure 1.7 Parent Tile (left) and Replaced Tile (right) [4] 8
Figure 1.8 Parent Tile (left) and Added Tile (right) [4] 9
Figure 1.9 glTF Object Hierarchy [7] 11
Figure 1.10 Binary glTF Layout [8] 19
Figure 1.11 Data layout of a Batched 3D Model [9] 21
Figure 1.12 Classic quadtree subdivision [6] 23
Figure 1.13 Quadtree with tight bounding volumes around each child [6] 23
Figure 1.14 Example in Canary Wharf in Cesium [6] 24
Figure 1.15 Quadtree with non-uniform and overlapping tiles [6] 24
Figure 1.16 Loose quadtree example [6] 25
Figure 1.17 K-d trees [6] 25
Figure 1.18 Traditional octree subdivision [6] 26
Figure 2.1 Partitioning Process Block Diagram 28
Figure 2.2 Use-Case Diagram 28
Figure 2.3 Sample Application Sequence Diagram 29
Figure 2.4 Statistic Octree Tiling [10] 30
Figure 2.5 Adaptive Octree Tiling 31
Figure 2.6 Screen of OSM on website [11] 31
Figure 2.7 Screen of Export in OSM [11] 32
Figure 2.8 Example of file .osm being rendered by OSM2World [12] 32
Figure 2.9 Some extracting function in OSM2World 33
Figure 3.1 Block Diagram Of Data Extraction Stage 34
Figure 3.2 Read Data Process for each Attribute 40
Figure 3.3 Update total Indices List and other Attribute's Lists 42
Figure 3.4 Calculate Root Bounding Volume 43
Figure 3.5 Block Diagram Of Tile Partitioning Stage 44
Figure 3.6 Example of level and xyz combination in URI [10] 45
Figure 3.7 Choosing Dividing Point in Static Octree Tiling 46
Figure 3.8 Choosing Dividing Point in Adaptive Octree Tiling 47
Figure 3.9 The block Diagram Of Tileset Complication Stage 52
Figure 3.10 Update Attributes for each Tile 54
Figure 3.11 Example of a children tile in the JSON tileset 60
Figure 4.1 Welcome Page 63
Figure 4.2 Rendering Page (Cesium JS) 63
Figure 4.3 Rendering Page with detailed Bounding Volume with statistics 64

Figure 4.4 Rendering Time For Maximum 2000 Triangles Per Tile 65
Figure 4.5 Result after rendering with Static Tiling (left) and Adaptive Tiling (right) 66
Figure 4.6 The Statistics with Static Tiling (left) and Adaptive Tiling (right) 66
Figure 4.7 Rendering Time For Maximum 3000 Triangles Per Tile 67
Figure 4.8 Result after rendering with Static Tiling (left) and Adaptive Tiling (right) 67
Figure 4.9 The Statistics with Static Tiling (left) and Adaptive Tiling (right) 68

LIST OF TABLES

Table 1.1 Data Type for componentType [7]..... 15
Table 1.2 Number of components for Type [7] 15
Table 3.1 Bounding Volume of each Tile based on the Root Bounding and Dividing Point 48

ABSTRACT

This thesis focuses on the computation of glTF files into tileset based on the standard of 3D Tiles, aiming to optimize the performance and efficiency of managing and rendering large-scale 3D datasets, hence upgrading the user experience. The conversion of glTF file into 3D Tiles is a critical step in organizing, streaming, and rendering complex 3D data.

This research investigates, proposes, and compares two specific partitioning methods for generating 3D Tiles form glTF files, which are Static partitioning scheme and Adaptive partitioning scheme. The proposed methods adjust the partitioning scheme based on factors such as the maximum number of triangles that a tile should contain, user's viewpoint or level of detail requirement. By adapting the partitioning scheme, the methods optimize the rendering process, resulting in improved performance and efficiency.

Experimental evaluations were conducted using real-world glTF datasets, which are extracted from Open Street Map, to assess the effectiveness of the proposed partitioning methods. The results demonstrated the difference of the methods in enhancing the performance and efficiency of glTF to 3D Tiles conversion.

The proposed adaptive partitioning method offers significant improvements in the management and rendering of large-scale 3D datasets converted from glTF files into 3D Tiles. By dynamically adjusting the partitioning scheme, these methods optimize rendering speed, reduce memory usage, and enhance visual quality. The experimental results validate the effectiveness of the adaptive partitioning methods, demonstrating their superiority over traditional fixed partitioning schemes.

INTRODUCTION

The rapid growth of 3D data poses challenges in managing and retrieving large-scale datasets. The 3D Tiles specification by Cesium offers a powerful framework for streaming and visualizing massive 3D datasets on the web. To optimize data management and retrieval, efficient partitioning methods are crucial. These techniques divide large 3D models into smaller tiles, enhancing performance and scalability for seamless exploration of complex 3D scenes.

In this thesis we focus on exploring and evaluating different partitioning methods tailored for 3D Tiles, with a specific emphasis on leveraging the glTF format as the underlying data source. glTF, the runtime asset format for WebGL, represents 3D models in a compact and interoperable manner, making it an ideal candidate for storing and transmitting 3D data within the 3D Tiles ecosystem.

The primary objective of this research is to investigate various partitioning strategies and evaluate their effectiveness in terms of data organization, loading time, rendering efficiency.

In this thesis, Chapter 1, the Theoretical Basis, serves as the foundation for understanding the fundamental concepts of 3D Tiles and glTF, glb and b3dm. This chapter provides an overview of the 3D Tiles specification, explaining its purpose, structure, and the benefits it offers for streaming and visualizing geospatial data. Additionally, it explores the structure and characteristics of the glTF, glb and b3dm formats, detailing how 3D models and their associated metadata are stored within the glTF file, and how glb and b3dm are used as the main component in tileset for streaming. Understanding the theoretical basis of 3D Tiles and glTF is essential for developing effective partitioning methods that leverage the strengths of these technologies.

Chapter 2, Partitioning Methods Approach, focuses on the objectives and the requirement of the project, which leads to the choice of partitioning algorithms. By identifying the specific goals and necessities of the project, this chapter lays the groundwork for determining the most suitable techniques for dividing the large 3D model into manageable tiles.

Chapter 3, Algorithm Development, provides a detailed account of the design and formulation of the partitioning algorithms introduced in Chapter 2. This chapter explains the underlying principles and methodologies behind each algorithm, exploring their strengths and limitations.

Chapter 4, Implement & Measurement, delves into the practical implementation and performance evaluation of the developed partitioning methods. This chapter

discusses the benefits and trade-offs of each algorithm, considering factors such as loading time, rendering efficiency. By conducting thorough measurements and comparisons, this chapter provides empirical evidence of the efficiency and effectiveness of the partitioning methods, helping to identify the most suitable approach for different use cases.

Through this thesis, we aim to advance the field of 3D geospatial data management and retrieval within the 3D Tiles framework. By exploring and developing partitioning methods based on the glTF format, we seek to enhance the efficiency, scalability, and user experience of 3D geospatial applications. The findings and insights gained from this research will contribute to the body of knowledge in the field, providing practical guidelines and recommendations for implementing partitioning strategies that optimize data management and retrieval in large-scale 3D geospatial environments.

CHAPTER 1. THEORETICAL BASIS

This chapter serves as the foundation for understanding the inspiration and the fundamental concepts of 3D Tiles, glTF, glb, and b3dm. It provides an overview of these technologies, explaining their purposes, structures, and how they contribute to efficient management and retrieval of 3D geospatial data. Moreover, it will propose the methods of partitioning that the community has developed.

1.1 3D Tiles Specifications

3D Tiles is an open standard data format specifically designed for streaming and rendering large-scale, geospatial 3D datasets in real-time. It was developed by the Cesium team at AGI (Analytical Graphics Inc.) and is widely adopted in the geospatial industry for visualizing and analyzing 3D data on the web, desktop, and mobile platforms.

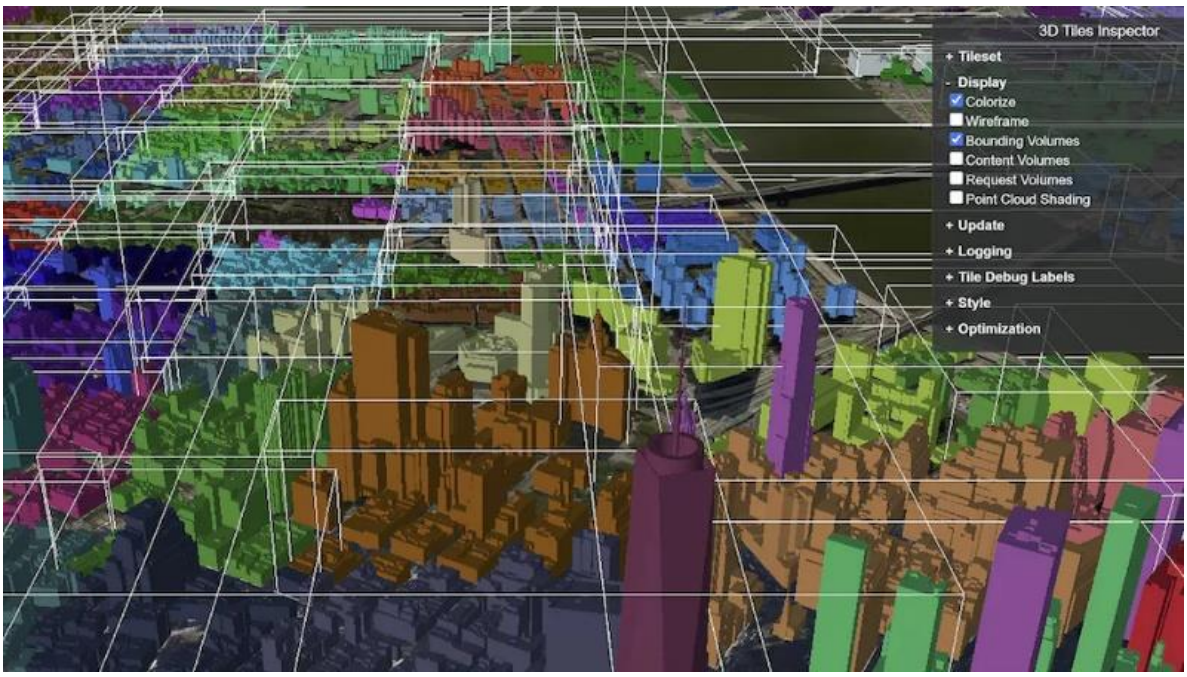

Figure 1.1 Sample of an application using 3D Tiles [2]

The fundamental idea behind 3D Tiles is to enable efficient transmission and rendering of massive 3D datasets, such as terrain, buildings, point clouds, and other geospatial information. It achieves this goal by utilizing a hierarchical data structure that organizes the data into a set of tiles, each representing a smaller portion of the overall scene. These tiles are designed to be streamed and loaded on-demand, allowing applications to focus on rendering only the necessary data at a given moment, optimizing both performance and memory usage.

Due to the open standard nature of 3D Tiles and its ability to efficiently handle and display massive 3D datasets, it has become a preferred format for visualizing and analyzing geospatial data. Many companies, particularly those involved in areas such as geospatial mapping, urban planning, virtual and augmented reality, and defense and intelligence, have integrated 3D Tiles into their workflows and applications.

As introduced in the main page of Cesium [3], 3D Tiles have some specific features:

- *Open:* 3D Tiles are an open specification with an open-source implementation in Cesium.
- *Optimized for streaming and rendering:* 3D Tiles were created to enhance the streaming and rendering performance of large and diverse datasets. It utilizes a spatial data structure and hierarchical level of detail to stream only the visible and relevant tiles for a given 3D view. The tile payloads are compressed using context-aware techniques like Open3DGC or oct-encoding. The design for 3D Tiles is streamlined for fast and simple rendering with WebGL, minimizing client-side processing. Tiles can be pre-batched or batched on-the-fly to reduce the number of WebGL draw calls and optimize performance.
- *Designed for 3D:* 3D Tiles are designed for true 3D experiences, not limited to 2.5D perspectives. They use geometric error and tunable pixel error for Level-Of-Detail (LOD) selection, allowing flexibility in performance and visual quality across multiple zoom levels. Unlike 2D tiling schemes based on Web Mercator projection, 3D Tiles employ adaptable tiling schemes in all three dimensions to accommodate the distribution of 3D models in the dataset. In addition to traditional geospatial features like polygons and polylines, 3D Tiles support full 3D models with meshes, materials, and node hierarchies.
- *Interactive & Styleable:* 3D Tiles support interactive selection and styling. Even with WebGL optimizations such as batching, 3D Tiles allow for individual model interaction such as highlighting on mouseover or removing a 3D building. Tiles can contain metadata for each model to allow additional interaction, such as querying third-party web services using a building ID.
- *Adaptable:* 3D Tiles offer adaptive spatial subdivision in 3D, using various spatial data structures like k-d trees, quadtrees, octrees, and grids. Unlike traditional quadtree subdivision, 3D Tiles enable flexible and efficient subdivision based on rendering cost and model distribution. A generic runtime engine like Cesium can work with any subdivision technique.
- *Flexible:* In traditional 2D map tiles, zooming in involves replacing lower-resolution tiles with higher-resolution ones, known as replacement refinement.

However, for 3D datasets like buildings and point clouds, a more flexible approach is needed. 3D Tiles support both replacement and additive refinement, where only new data is streamed instead of downloading the same content multiple times. Additive refinement also allows rendering of child tiles as they are downloaded, providing a performance advantage over replacement refinement.

- *Heterogeneous:* 3D Tiles embrace heterogeneity as different types of 3D datasets require unique representations. Batched models, instanced models, and point clouds, for example, each demand distinct approaches. 3D Tiles facilitate this heterogeneity through adaptive subdivision, flexible refinement, and support for various tile formats. This versatility allows for discrete levels of detail and hierarchical levels of detail (HLOD). For instance, a 3D building could be represented as a billboard and label at a lower LOD, an extruded footprint at a higher LOD, a 3D model at the next level, and a textured 3D model at the highest LOD.

Below are some key components with detailed explanation of 3D Tiles Specifications [4].

1.1.1. Tileset

The core element of 3D Tiles is the tileset. A tileset is a set of tiles, organized in a hierarchical structure. The tileset is described in JSON (JavaScript Object Notation). The following is a simple example of a tileset, introducing the most important concepts and elements.

```
{
    "asset": {...
    },
    "properties": {...
    },
    "geometricError": 100,
    "root": {
        "geometricError": 20,
        "boundingVolume": {
            "region": [...
            ]
        },
        "transform":[...],
        "refine": "ADD",
        "children": [
            {
                "geometricError": 10,
                "boundingVolume": {...
                },
                "content": {
                    "uri": "sample.b3dm"
                }
            }}
  }
```

The main tileset JSON contains a basic description of the *asset* and general *properties*, such as the name of creator of the version of that tileset. The *geometricError* is used to determine when the root tile should be rendered. Each tile contains a *boundingVolume* that enclosed the content of the tile and all the children. It also contains a *geometricError* that determines when the children should be rendered.

In the above example tileset, the children tiles refer to 3D models in the format of Batched 3D Model (Section 1.4). The child tiles are rendered when a level of detail is requested. In this example, the contents of the child tiles will be added to the low-level representation of the root node. Alternatively, the child tiles could contain more detailed models that replace the low-level model. Section 1.1.5 will inform two *refinement strategies* that are supported by 3D Tiles.

Different tile formats can be combined in one tileset: Batched 3D Model, Point Cloud, Instanced 3D Model,... When the tileset is rendered, the tiles are combined, at the appropriate level of detail, to generate the final rendered result.

1.1.2. Tileset and Tiles

A tileset is a set of tiles, which are organized as a hierarchical data structure, called a tree. The tileset itself contains the root tile, and each tile may have child tiles. A tile can also refer to another tileset. This makes it possible to combine tilesets flexibly and hierarchically into larger ones. Each tile may refer to renderable content. This content can have different formats and may represent, for example, textured terrain surfaces, 3D models, or point clouds.

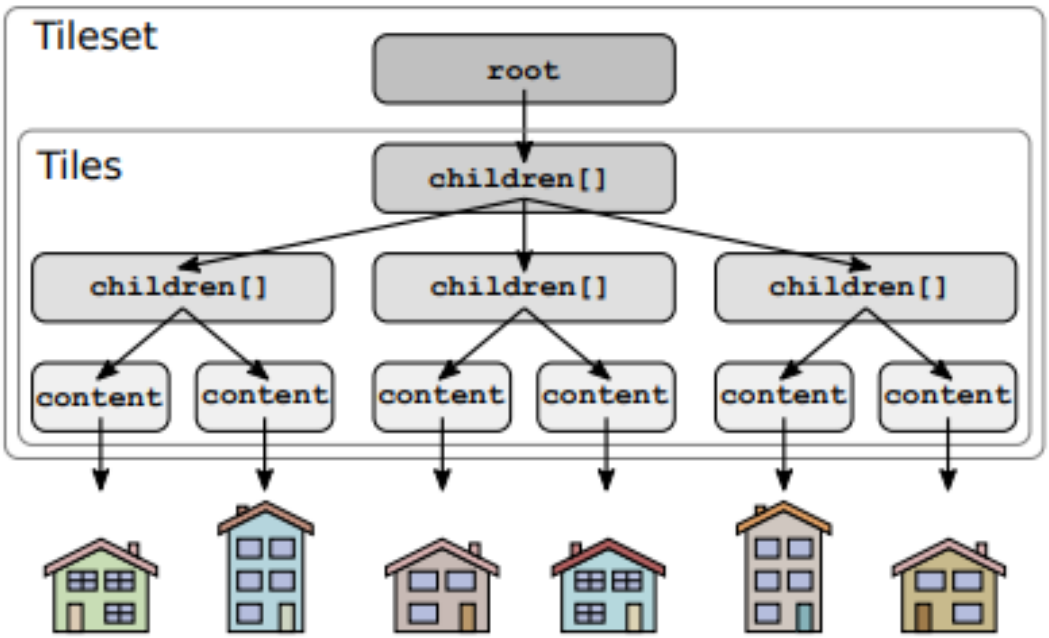

Figure 1.2 Hierarchical structure of a Tileset [4]

Both tileset and tiles are described in JSON. The tileset JSON file contains basic information about the tileset itself, and the description of the tiles. Below are the detailed properties of both Tileset and Tile.

Tileset properties:

- *Root tile:* The *root* property of a tileset is a tile that represents the root of the tile hierarchy.
- *Geometric error:* The *geometricError* property is used to quantify the visual error that could occur if the tileset was not rendered. When the visual error exceeds a certain threshold, then the tileset and the tiles that it contains are considered for rendering. Details of how the geometric error is used are given in Section 1.1.4, "Geometric Error."
- *Property:* The renderable content of tiles may have associated properties. For example, when the tiles contain buildings, then the height of each building can be stored in the tile. The *properties* object of a tileset contains the minimum and the maximum of the select property values for all tiles in the tileset.
- *Metadata:* Information about the 3D Tiles version and application-specific version information can be stored in the asset property of a tileset.

Tile properties:

- *Content:* The actual renderable content that is associated with a tile is referred to via a URI in the *content* property.
- *Children:* The hierarchical structure of tiles is modeled as a tree: each tile may have children, which are tiles in an array name *children*.
- *Bounding Volume:* Each tile has an associated bounding volume, and different types of bounding volumes may be stored in the *boundingVolume* property. The possible types of bounding volumes are presented in Section 1.1.3, “Bounding Volume”. Each bounding volume encloses the content of the tile and the content of all children, yielding a spatially coherent hierarchy of bounding volumes.
- *Geometric error:* The renderable content of tiles may have different levels of detail. For tiles, the *geometricError* property quantifies the degree of simplification of the content in the tile compared to the highest level of detail. The geometric error is used as described in Section 1.1.4, "Geometric Error", to determine when the child tiles should be considered for rendering.
- *Refinement strategy:* When the visual error of a tile with a certain level of detail exceeds a threshold, then the child tiles are considered for rendering. The way the additional detail from the child tiles is incorporated into the rendering process is determined by the *refine* property. The different refinement strategies are explained in Section 1.1.5, "Refinement".

1.1.3. Bounding Volume

In 3D Tiles, a tileset consists of a collection of tiles organized in a hierarchical manner, with each tile having its own bounding volume. This hierarchical structure forms a spatial data organization that optimizes rendering and enables efficient spatial queries. Furthermore, each tile contains renderable content that is also associated with a bounding volume. Unlike the tile's bounding volume, the content bounding volume tightly fits around the content itself. This content bounding volume is used for visibility queries and view frustum culling, enhancing rendering performance. 3D Tiles support various types of bounding volumes, allowing the selection of the most appropriate type based on the data's structure and characteristics.

Type of Bounding Volume:

- *Sphere:* A *bounding sphere* is a simple bounding volume that allows easy and efficient intersection tests. It is defined by its center position and the radius.

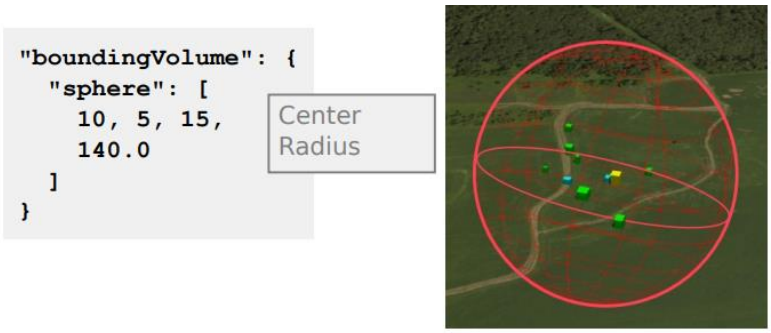

Figure 1.3 Sphere Bounding example [4]

- *Box:* An *oriented bounding box* fits the bounding volume more tightly to the geometry, particularly for regular, technical structures like CAD models. It is defined by the center position of the box, and three 3D vectors. The vectors define the directions and half-lengths of the x, y and z-axis.

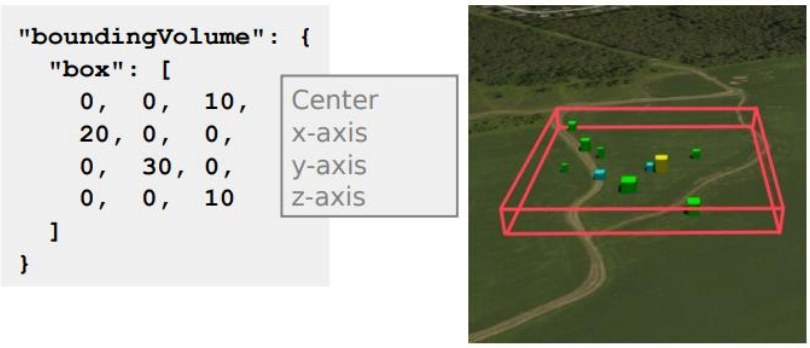

Figure 1.4 Box Bounding example [4]

- *Region:* A region is particularly well suited to geographical information systems because the sides of the region are defined with latitude and longitude coordinates, for the west, south, east and north of the region, and a minimum

and maximum height. Latitudes and longitudes are in the WGS 84 [2] datum as defined in EPSG 4979 and are in radians. The minimum and maximum height are given in meters above or below the WGS 84 ellipsoid.

Figure 1.5 Region Bounding example [4]

1.1.4. Geometric Error

One of the goals of 3D Tiles is to efficiently stream massive datasets to runtime engines, and still allow the runtime to render this content efficiently. The hierarchical structure of tiles in a tileset therefore incorporates the concept of a Hierarchical Level of Detail (HLOD): tiles at the top of the hierarchy contain representations of the renderable content with a low level of detail. The child tiles contain content with a higher level of detail. The rendering runtime can dynamically choose the level of detail that offers the best trade-off between performance and rendering quality.

The key to determining which level of detail should be rendered is geometric error. Each tileset and each tile has a *geometricError* property that quantifies the error of the simplified geometry compared to the actual geometry. The runtime translates this geometric error into a screen-space error (SSE). The SSE quantifies how much of the geometric error will be visible, in terms of pixels on the screen.

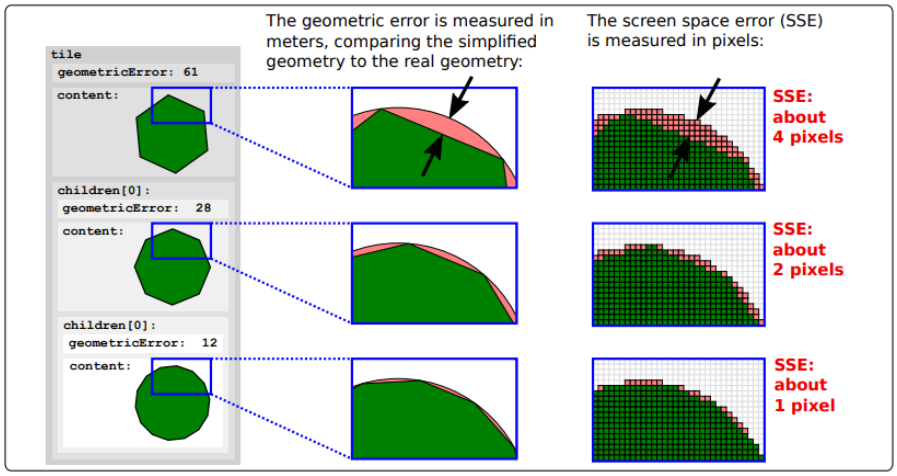

Figure 1.6 Example of how geometricError effect the visible level on the screen [4]

When the SSE exceeds a certain threshold, the runtime will render a higher level of detail. For a tileset, the geometric error is used to determine whether the root tile

should be rendered. For a tile, the geometric error is used to determine whether the children of the tile should be rendered.

To compute the actual SSE, the runtime must be based on the given geometricError and the view configuration, which is the position and orientation of the viewer, and on the resolution at which the final image will be rendered. One possible way of computing the SSE is as follows [4]:

$$SSE = \frac{geometricError\ .\ screenHeight}{tileDistance\ .\ 2\ .\tan\left(\frac{fovy}{2}\right)} \qquad \text{Equation 1.1}$$

Where *geometricError* is the geometric error that was stored in the tileset or tile, *screenHeight* is the height of the rendering screen in pixels, *tileDistance* is the distance of the tile from the eye point, and *fovy* is the opening angle of the view frustum in the y-direction.

1.1.5. Refinement

When a tileset is rendered, the runtime traverses the hierarchy of tiles, examines the geometric error of each tile, and computes the associated screen space error. When this screen space error exceeds a certain threshold, then the runtime will recursively consider the child tiles for rendering. The child tiles that contain renderable content have a higher level of detail and a smaller screen space error. The content of the child tiles can be used to increase the level of detail through one of two refinement strategies. The refinement strategy is determined by the *refine* property of a tile and can either be “REPLACE” or “ADD”.

- *Replace:* The child tiles contain a complete representation of the content with a higher level of detail. When the child tiles are rendered, this content will replace the content of the parent tile.

Figure 1.7 Parent Tile (left) and Replaced Tile (right) [4]

- *Add:* The child tiles contain additional details for the content of the parent tile. When the child tiles are rendered, this content will be rendered on top of the content of the parent tile.

Figure 1.8 Parent Tile (left) and Added Tile (right) [4]

The *refine* property can be given for each tile individually but is only required in the root tile of a tileset. When it is not given, the strategy will be inherited from the parent tile.

1.1.6. Transform in 3D Tiles

The renderable content in 3D Tiles may be given in different coordinate systems, so to convert between local coordinate systems, each tile has an optional *transform* property. This property is a column-major 4x4 affine transformation matrix and transforms the coordinate system of the tile into the coordinate system of the parent, defaulting to the identity matrix when it is not defined.

The transform matrix of a tile affects the positions, normals, and the bounding volumes of the tile and its content. The positions of features (like instances in an Instanced 3D Model, or points in a Point Cloud) are multiplied with the transform matrix to bring them from the local coordinate system into the coordinate system of the parent tile. The normals are multiplied with the top-left 3x3 matrix of the transpose of the inverse of the matrix, to properly take non-uniform scaling into account. The bounding volumes are transformed with the matrix, except the "region" bounding volumes, which are explicitly defined to be in EPSG:4978 coordinates.

EPSG:4978 is a coordinate reference system (CRS) used to represent 3D geodetic coordinates on the Earth's surface. It is also known as the "World Geodetic System 1984 (WGS 84) Geocentric" CRS. In EPSG:4978, the coordinates are expressed as a triplet of values latitude, longitude and height [5].

In this project, we use glTF file as the main source of data, so we need to use *transform* property since 3D tiles and glTF have different definitions for the up axis in their coordinate systems.

3D Tiles defines the z-axis as up for local Cartesian coordinate systems. In contrast, glTF considers the y-axis as up. When glTF assets are embedded in Batched 3D Models and Instanced 3D Models, these different conventions must be considered by transforming the glTF asset at runtime so that the z-axis points upwards. This is done by rotating the model about the x-axis by $\pi/2$ radians. Equivalently, apply the following matrix transform [6]:

```
    "transform":[
       1.0, 0.0, 0.0, 0.0,
       0.0, 0.0, -1.0, 0.0,
       0.0, 1.0, 0.0, 0.0,
       0.0, 0.0, 0.0, 1.0 ]
```

Additionally, the Cesium group has developed a function that utilizes a transformation matrix to convert the location data to the desired destination for display on the client side. This function takes longitude, latitude, and height as input parameters.

1.1.7. Tile Format Specifications

3D Tiles support different types of tile format, which contain the data of the steaming objects, namely as: Batched 3D Model (b3dm), Instanced 3D Model (i3dm), Point Cloud or Composite. In this project, we focus and use b3dm as the main type of format.

A tileset can contain any combination of tile formats.

1.2. GL Transmission Format

GL Transmission Format (glTF) is an open standard file format designed for efficient transmission and loading of 3D scenes and models. It is developed by the Khronos Group, a consortium of companies and organizations focused on creating open standards for graphics and computing.

glTF is specifically designed for real-time rendering and is widely used in applications such as 3D graphics, virtual reality (VR), augmented reality (AR), and game development. It aims to be lightweight, extensible, and optimized for web and mobile platforms.

Combined with 3D Tiles, the glTF file format is emerging as the primary choice for storing and transforming 3D data, paving the way for its seamless integration with the b3dm format.

A primary goal of glTF is to be deployable on a wide range of devices and platforms, including the web and mobile devices with limited processing and memory resources. glTF can be evolved, to keep pace with growing computing capabilities over time. This helps to foster broad industry consensus on 3D functionality that can be used ubiquitously, including Physically Based Rendering

glTF combines an easily parsable JSON scene description with one or more binary resources representing geometry, animations, and other rich data. These binary resources can often be loaded directly into GPU buffers with no additional parsing or

processing, combining the faithful preservation of full hierarchical scenes, nodes, meshes, cameras, materials, and animations with efficient delivery and fast loading.

The detailed of this file format are shown below [7].

1.2.1. Overview

A glTF asset is represented by:

- A JSON-formatted file (.gltf) containing a full scene description: node hierarchy, materials, cameras, as well as descriptor information for meshes, animations, and other constructs.
- Binary files (.bin) containing geometry, animation, and other buffer-based data.
- Image files (.jpg, .png) containing texture images.

Binary and image resources are also embedded directly in JSON using the *uri* or stored side-by-side with JSON in GLB container. In this project, the binary and image resources are embedded directly in the file.

1.2.2. Coordinate System and Units

glTF uses a right-handed coordinate system, which defines +Y as up, +Z as forward, and +X as left; the front of a glTF asset faces +Z.

The units for all linear distances are meters. All angels are in radians. Positive rotation is counterclockwise.

1.2.3. glTF Object Hierarchy

The figure below shows relations between top-level arrays in a glTF asset.

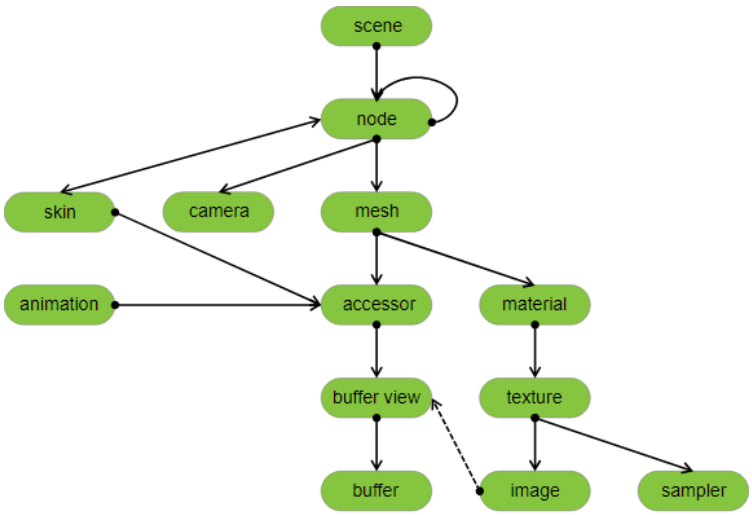

Figure 1.9 glTF Object Hierarchy [7]

The figure illustrates the main components that exist in glTF and how they are related to each other. The following sections will explain each part clearly.

1.2.4. Scenes

Scenes is the set of visual objects to render. Scenes are defined in a *scenes* array. All nodes listed in *scene.nodes* array are root nodes, and they will not be listed in a *node.children* array of any node.

The following example defines a glTF asset with a single scene that contains a single node.

```
{
    "nodes": [
        {
            "name": "singleNode"
        }
    ],
    "scenes": [
        {
            "name": "singleScene",
            "nodes": [
                0
            ]
        }
    ],
    "scene": 0
}
```

1.2.5. Nodes and Hierarchy

glTF assets define ***nodes***, which are the objects comprising the scene to render. These nodes may include *transform* properties, which serve a similar purpose as the *transform* in 3D Tiles. These properties allow for the modification of the coordinate system within the scene.

The node hierarchy is defined using a node's *children* property, as in the following example:

```
{
    "nodes": [
        {
            "name": "Car",
            "children": [1, 2]
        },
        {
            "meshes": 0
        },
        {
            "meshes": 1
        }
    ]
}
```

The node named "Car" above has four children. Each of those nodes could in turn have its own children, creating a hierarchy of nodes.

1.2.6. Buffers and Buffer Views

A ***buffer*** is arbitrary data stored as a binary blob. The buffer contains the combination of geometry, animation, skins and images. Binary blobs allow efficient creation of GPU buffers and textures since they require no additional parsing, except perhaps decompression.

All buffer data defined in this specification must use little endian byte order.

The following example defines a buffer. The *byteLength* property specifies the size of the buffer file. The *uri* property is the URI to the buffer data.

```
{
   "buffers": [
       {
           "byteLength": 102040,
           "uri": "duck.bin"
       },
       {
           "byteLength": 102040,
           "uri":"data:application/gltfbuffer; base64,J/FFwzMzE0AncfdCwYpF"

       }
   ]
}
```

glTF assets may have any number of buffer resources. Buffers are defined in the asset's *buffers* array. As in the above example, the buffer data may alternatively be embedded in the glTF file via *data*, which is URI with base64 encoding. When *data* is used for buffer storage, its *mediatype* field must be set to "application/octet-stream" or "application/gltf-buffer".

A ***buffer view*** represents a contiguous segment of data in a buffer, defined by a byte offset into the buffer specified in the *byteOffset* property and a total byte length specified by the *byteLength* property of the buffer view.

Buffer views used for images, vertex indices, vertex attributes, or inverse bind matrices must contain only one kind of data, i.e., the same buffer view must not be used both for vertex indices and vertex attributes.

When a buffer view is used by vertex indices or attribute accessors it should specify the buffer view target with a value of element array buffer or array buffer respectively.

The following example defines two buffer views: the first holds the indices for an indexed triangle set, and the second holds the vertex data for the triangle set.

```
{
    "bufferViews": [
        {
            "buffer": 0,
            "byteLength": 25272,
            "byteOffset": 0,
            "target": 34963
        },
        {
```

```
            "buffer": 0,
            "byteLength": 76768,
            "byteOffset": 25272,
            "byteStride": 32,
            "target": 34962
        }
    ]
}
```

When a buffer view is used for vertex attribute data, it may have a *byteStride* property. This property defines the stride in bytes between each vertex. Buffer views with other types of data must not define *byteStride*.

Buffers and buffer views do not contain type information. They simply define the raw data for retrieval from the file. Objects within the glTF asset (meshes, skins, animations) access buffers or buffer views via ***accessors***.

1.2.7. Accessors

1.2.7.1 Overview

All binary data for meshes, skins and animations are stored in buffers and retrieved via ***accessors***.

An *accessor* defines a method for retrieving data as typed arrays from within a buffer view. The accessor specifies a component type (e.g., *float*) and a data type (e.g., VEC3 for 3D vectors), which when combined define the complete data type for each data element. The number of elements is specified using the *count* property. Elements could be, e.g., vertex indices, vertex attributes, animation keyframes, etc.

The *byteOffset* property specifies the location of the first data element within the referenced buffer view. If the accessor is used for vertex attributes (i.e., it is referenced by a mesh primitive or its morph targets), the locations of the subsequent data elements are controlled by the *bufferView.byteStride* property. If the accessor is used for any other kind of data (vertex indices, animation keyframes, etc.), its data elements are tightly packed.

All accessors are stored in the asset's *accessors* array.

The following example shows two accessors, the first is a scalar accessor for retrieving a primitive's indices, and the second is a 3-float-component vector accessor for retrieving the primitive's position data.

```
{
    "accessors": [
        {
            "bufferView": 0,
            "byteOffset": 0,
            "componentType": 5123,
            "count": 12636,
            "max": [
                4212
            ],
            "min": [
```

```
                0
            ],
            "type": "SCALAR"
        },
        {
            "bufferView": 1,
            "byteOffset": 0,
            "componentType": 5126,
            "count": 2399,
            "max": [
                0.961799,
                1.6397,
                0.539252
            ],
            "min": [
                -0.692985,
                0.0992937,
                -0.613282
            ],
            "type": "VEC3"
        }
    ]
}
```

In the example, there are *componentType* with value of 5126 and the *type* with value of "VEC3", those number and definitions are used to define the element size, in bytes, which is equals to the multiplication of the size in bytes of the *componentType* with the number of components defined by *type*.

1.2.7.2 Accessor Data Types

The two following tables are the corresponding values of *componentType* and *type*.

Table 1.1 Data Type for componentType [7]

componentType	**Data Type**	**Signed**	**Bytes**
5120	Signed byte	Signed, two's complement	1
5121	Unsigned byte	Unsigned	1
5122	Signed short	Signed, two's complement	2
5123	Unsigned short	Unsigned	2
5125	Unsigned int	Unsigned	4
5126	Float	Signed	4

Table 1.2 Number of components for Type [7]

Type	SCALAR	VEC2	VEC3	VEC4	MAT2	MAT3	MAT4
Number of components	1	2	3	4	4	9	16

So based on the two tables, *componentType*, which is 5126, has the value of float, so each component is four bytes. The *type* is “VEC3”, so there are three components. The size of each element is 12 bytes (4 * 3). Thus, the accessor takes 28788 bytes (size of each element * *count* of accessor) in the range of the buffer view.

1.2.7.3 Accessors Bounds

Additionally, ***min*** and ***max*** properties are arrays hat contain per-component minimum and maximum values, respectively. The length of these arrays is equal to the number of accessor’s components.

Values stored in glTF JSON must match the actual minimum and maximum binary values stored in buffers.

1.2.8. Meshes

Meshes are defined as arrays of *primitives*. Primitives correspond to the data required for GPU draw calls. Primitives specify one or more *attributes*, corresponding to the vertex attributes used in the draw calls. Indexed primitives also define an *indices* property. Attributes and indices are defined as references to accessors containing corresponding data. Each primitive may also specify a *material* and a *mode* that corresponds to the GPU topology type (e.g., triangle set).

The following example defines a mesh containing one indexed triangle primitive:

```
{
    "meshes": [
        {
            "primitives": [
                {
                    "attributes": {
                        "NORMAL": 2,
                        "POSITION": 1,
                        "TEXCOORD_0": 4
                    },
                    "indices": 0,
                    "material": 3,
                    "mode": 4
                }
            ]
        }
    ]
}
```

Each attribute is defined as a property of the *attributes* object. The name of the property corresponds to an enumerated value identifying the vertex attribute, such as POSITION. The value of the property is the index of an accessor that contains the data.

Descriptions for each attribute semantic property are defined below:

- indices: the index to identify the position in POSITION list that create triangles

- POSITION: Unitless XYZ vertex positions
- NORMAL: Normalized XYZ vertex normals
- TEXCOORD_0: Texture coordinates

1.2.9. Getting Data of Attributions

Each attribute will have an index that points to a specific *accessor* with specific data (as shown in Section 1.2.7). Each accessor will have its own *bufferView*, *byteOffset*, *type*, *componentType* that represent for its attribution (as shown in Section 1.2.6). Moreover, the value of *bufferView* points to the buffer View located in the corresponding position in the *bufferViews* array. Finally, with the index of the *buffer* in the *bufferView*, we can get the data that has been encoded with base64 in the *buffers* array (as shown in Section 1.2.6).

If you want to get information of list of an attribute, for example a list of indices, then in the first stage, you must go through the above steps to point to the location of *buffer* that corresponds to the indices. After decoding the buffer data, you can use this step to get the specific data to get the information:

- Calculate **ByteStride**: which is equal to the size of each element as shown in Section 1.2.7.2.
- Calculate **Offset**: the total *offset = accessor.byteOffset + bufferView.byteOffset*
- Get the list: using the data that just decoded, with using the combination of *offset* (to specify the starting point) and *ByteStride* (to determine the size of each element, which will be used to update the index after each call) to get the data. In this step we should use loop function with the maximum number equals to *count* in the *accessor*.

1.2.10. Texture and Material Data

Textures are stored in the asset's *texture* array. A texture is defined by an image index, denoted by the *source* property and a sampler index (*sampler*). For example:

```
{
    "textures": [
        {
            "sampler": 0,
            "source": 2
        }
    ]
}
```

Samplers are stored in the *samplers* array of the asset. Each sampler specifies filtering and wrapping modes.

glTF defines *materials* using a common set of parameters that are based on widely used material representations from Physically Based Rendering (PBR). Specifically, glTF uses the metallic-roughness material model. Using this declarative representation of materials enables a glTF file to be rendered consistently across platforms.

All parameters related to the metallic-roughness material model are defined under the *pbrMetallicRoughness* property of *material* object. The following example shows how to define a gold-like material using the metallic-roughness parameters:

```
{
    "materials": [
        {
            "name": "gold",
            "pbrMetallicRoughness": {
                "baseColorFactor": [ 1.000, 0.766, 0.336, 1.0 ],
                "metallicFactor": 1.0,
                "roughnessFactor": 0.0
            }
        }
    ]
}
```

The metallic-roughness material model is defined by the following properties:

- *Base color:* The base color of the material
- *Metalness:* The Metalness of the material; values range from 0.0 (non-metal) to 1.0 (metal)
- *Roughness*: the roughness of the material; values range from 0.0 (smooth) to 1.0 (rough)

The value for each property may be defined using factors and/or textures (e.g., *baseColorTexture* and *baseColorFactor*). If a texture is not given, all respective texture components within this material model must be assumed to have a value of 1.0. If both factors and textures are present, the factor value acts as a linear multiplier for the corresponding texture values. A texture binding is defined by an *index* of a *texture* object and an optional index of texture coordinates.

1.3. glTF Binary File Format Specification

1.3.1. Overview

glTF provides two delivery options that can be used together:

- glTF JSON points to external binary data (geometry, key frames, skins) and images.
- glTF JSON embeds base64-encoded binary data, and images inline using data URIs.

Hence, loading glTF files usually requires either separate requests to fetch all binary data, or extra space due to base64-encoding. Base64-encoding requires extra processing to decode and increases the file size (by ~33% for encoded resources). While transport-layer gzip mitigates the file size increase, decompression and decoding still add significant loading time.

To avoid this file size and processing overhead, a container format, *Binary glTF* (.glb) is introduced that enables a glTF asset, including JSON, buffers, and images, to be stored in a single binary blob.

A Binary glTF asset can still refer to external resources. For example, an application that wants to keep images as separate files may embed everything needed for a scene, except images, in a Binary glTF.

The glb format is widely used in various applications and platforms due to its compact size and easy loading process. It is especially popular for web-based 3D applications and games, as it reduces the number of HTTP requests required for loading the 3D content, leading to faster loading times and improved performance. Additionally, glb files support lossless compression, further reducing their size while retaining the model's fidelity.

1.3.2. Structure

A Binary glTF (which can be a file, for example) has the following structure:

- A 12-byte preamble, called the *header*.
- One or more *chunks* that contain JSON content and binary data.

The *chunk* containing JSON may refer to external resources as usual and may also reference resources stored within other chunks.

1.3.3. Layout

Binary glTF is little endian. The figure below shows an example of a Binary glTF asset.

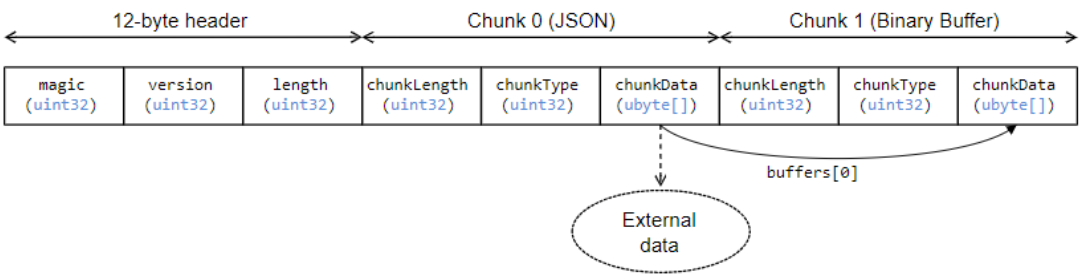

Figure 1.10 Binary glTF Layout [8]

The 12-byte header consists of three 4-byte entries:

- *magic:* must be equal to equal 0x46546C67. It is ASCII string glTF can be used to identify data as Binary glTF.
- *version:* indicates the version of the binary glTF container format. This specification defines version 2.
- *length:* is the total length of the binary glTF, including *header* and all *chunks*, in bytes.

Each chunk has the following structure:

- *chunkLength:* is the length of *chunkData*, in bytes.
- *chunkType:* indicates the type of chunk.
- *chunkData:* is the binary payload of the chunk.

Chunk 0 holds the glTF JSON, as it would be provided within a .gltf file. This chunk must be the very first chunk of a Binary glTF asset. By reading this chunk first, an implementation can progressively retrieve resources from subsequent chunks. This way, it is also possible to read only a selected subset of resources from a Binary glTF asset.

Chunk 1 contains the binary payload for geometry, animation key frames, skins, and images. This chunk must be the second chunk of the Binary glTF asset, and it must be padded with trailing zeros (0x00) to satisfy alignment requirements. When the binary buffer is empty or when it is stored by other means, this chunk should be omitted.

1.3.4. glTF to GLB Transformation

Cesium has created a tool called “gltf-pipeline” to convert file format glTF to GLB. This can be downloaded as a package in Node.js and using command in terminal for immediate transformation. Below is a sample command used for running this tool.

```
// download gltf-pipeline
npm install -g gltf-pipeline

// run command for convert gltf to glb
gltf-pipeline -f -i ${inputFile} -o ${outputFile}
```

1.4. Batched 3D Model

1.4.1. Overview

Batched 3D Model (B3DM) is an extension of the glTF specification, which is a popular format for 3D models and scenes. The B3DM format is designed to efficiently store and stream 3D models with geographic information, allowing for hierarchical structures and a batched representation.

A B3DM file can contain multiple 3D models, and each model is typically represented as a glTF asset, which includes mesh data, textures, and other necessary resources. B3DM files use a binary format, making them more compact and efficient for transmission over the internet.

B3DM files are widely used in applications that require streaming and rendering of large-scale 3D geospatial data, such as virtual globes, 3D mapping platforms, and geospatial visualization tools. They enable efficient visualization of complex 3D scenes with various levels of detail, making them an essential part of modern geospatial data visualization technologies.

Efficiency comes from transferring multiple models in a single request and rendering them in the least number of WebGL draw calls necessary. Using the core 3D Tiles spec language, each model is a *feature*.

1.4.2. Layout

A tile is composed of two sections: a header immediately followed by a body. The following figure shows the Batched 3D Model layout.

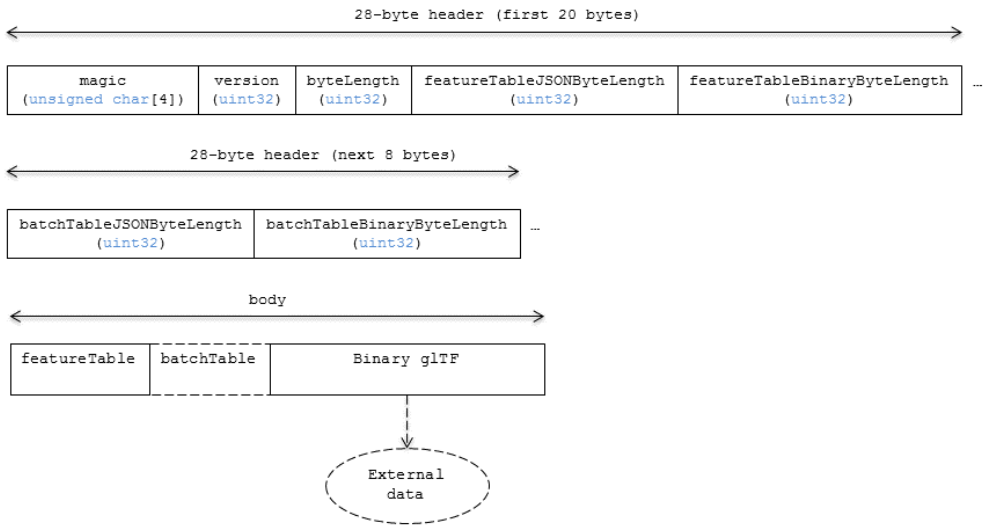

Figure 1.11 Data layout of a Batched 3D Model [9]

The 28-byte header contains the following fields:

- *magic*: This can be used to identify the content as a Batched 3D Model tile.
- *version*: The version of the B3DM format. It is currently 1.
- *byteLength*: The length of the Feature Table JSON section in bytes
- *featureTableJSONByteLength*: The length of the Feature Table JSON section in bytes

- *featureTableBinaryByteLength*: The length of the Feature Table binary section in bytes.
- *batchTableJSONByteLength*: The length of the Batch Table JSON section in bytes. Zero indicates there is no Batch Table.
- *batchTableBinaryByteLength*: The length of the Batch Table binary section in bytes. If *batchTableJSONByteLength* is zero, this will also be zero.

The body section immediately follows the header section, and is composed of three fields: *Feature Table*, *Batch Table*, *Binary glTF*.

Feature Table contains values for b3dm semantics, while *Batch Table* contains per-model applications-specific properties, indexable by *batchId*, that can be used for declarative styling and application-specific use cases such as populating a UI or issuing a REST API request. In the binary glTF section, each vertex has a numeric *batchId* attribute in the integer range [0, number of models in the batch -1]. The *batchId* indicates the model to which the vertex belongs. This allows models to be batched together and still be identifiable.

The *Binary glTF* immediately follows the Feature Table and Batch Table. It may embed all its geometry, texture, and animations, or it may refer to external sources for some or all these data.

1.4.3. Convert GLB to B3DM

Cesium uses b3dm as the main format to contain data for streaming in 3D Tiles. To get b3dm format we can use the tool that Cesium group has created called “3d-tiles-tools”. The specific command to convert is shown below.

```
// download 3d-tiles-tools
npm install 3d-tiles-tools

//convert glb to b3dm
npx 3d-tiles-tools glbToB3dm -i ${inputPathFile} -o ${outputPathFile}
```

1.5. Sample Tiling Methods

1.5.1. Quadtrees Tiling

A quadtree is created when each tile has four uniformly subdivided children (e.g., using the center latitude and longitude), similar to typical 2D geospatial tiling schemes. Empty child tiles can be omitted.

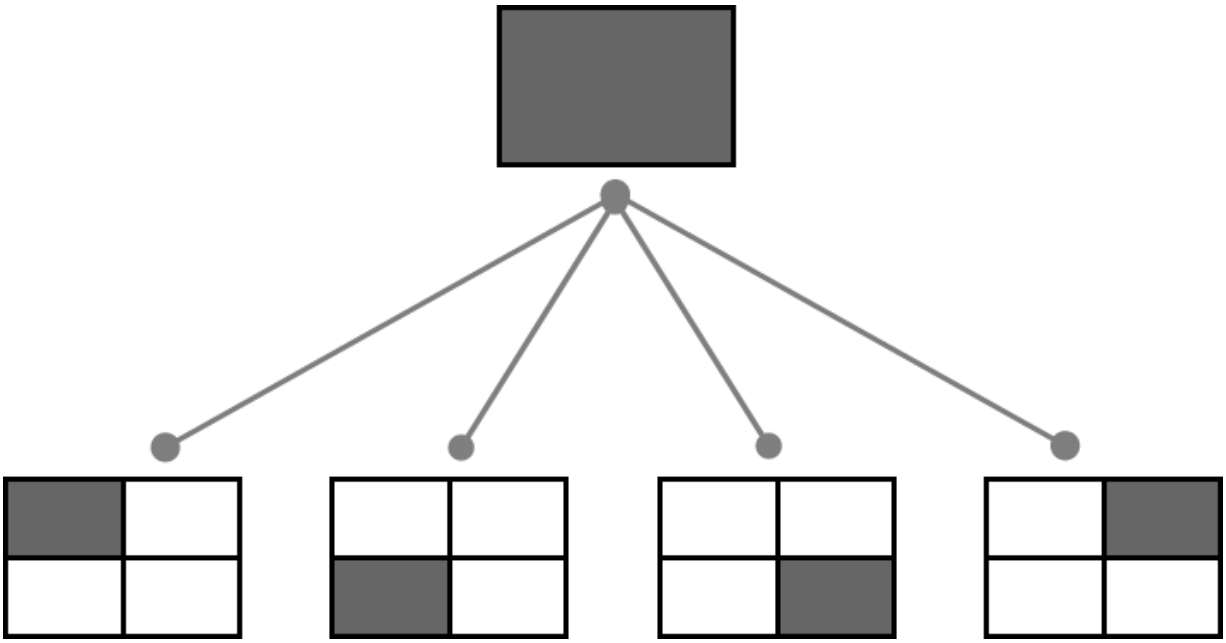

Figure 1.12 Classic quadtree subdivision [6]

3D Tiles enable quadtree variations such as non-uniform subdivision and tight bounding volumes (as opposed to bounding, for example, the full 25% of the parent tile, which is wasteful for sparse datasets).

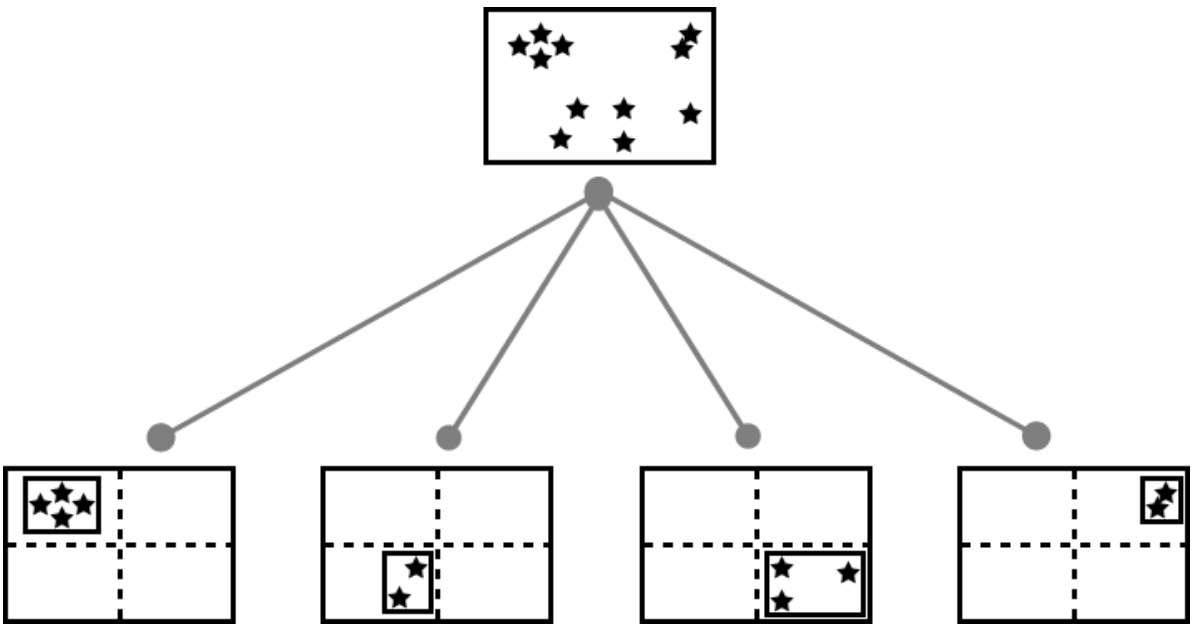

Figure 1.13 Quadtree with tight bounding volumes around each child [6]

For example, here is the root tile and its children for Canary Wharf. Note the bottom left, where the bounding volume does not include the water on the left where no buildings will appear:

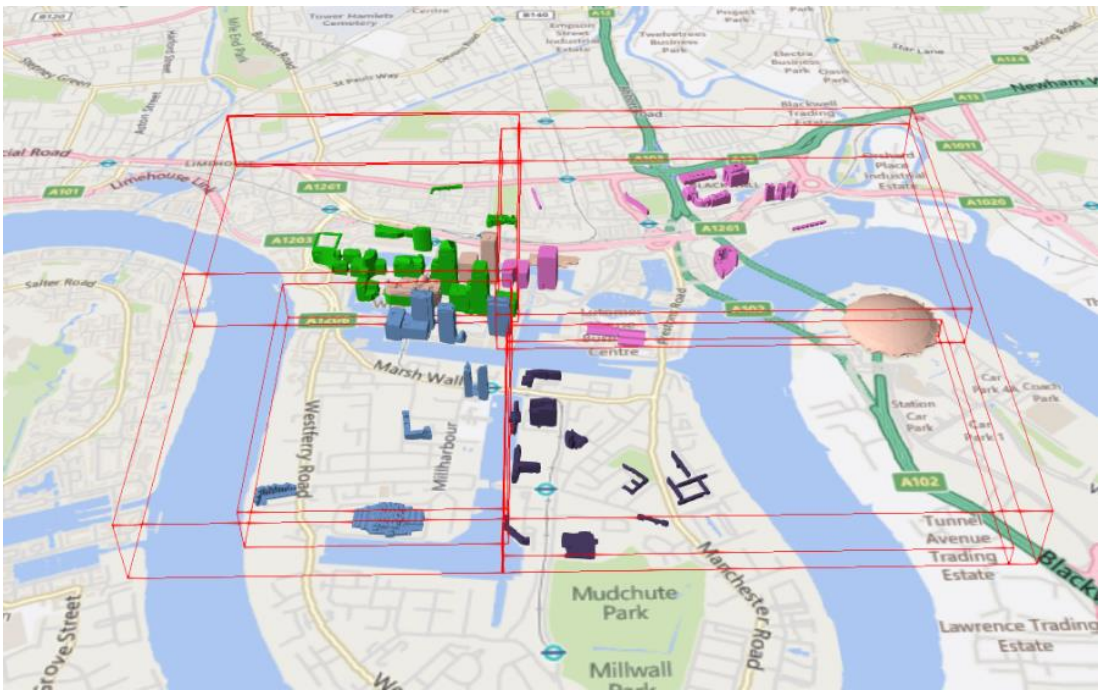

Figure 1.14 Example in Canary Wharf in Cesium [6]

3D Tiles also enable other quadtree variations such as loose quadtrees, where child tiles overlap but spatial coherence is still preserved, i.e., a parent tile completely encloses all of its children. This approach can be useful to avoid splitting features, such as 3D models, across tiles.

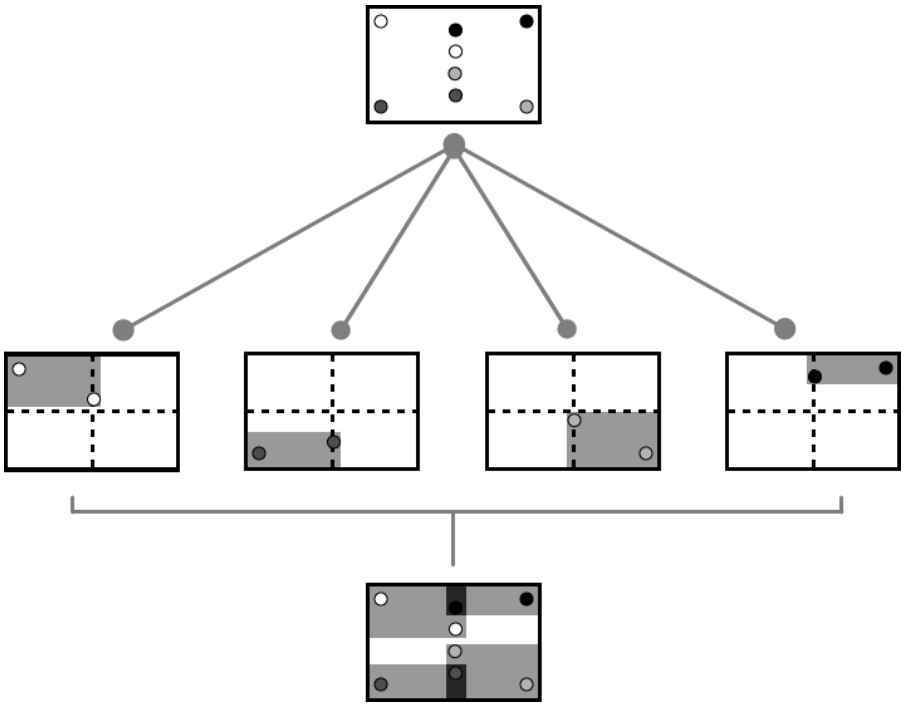

Figure 1.15 Quadtree with non-uniform and overlapping tiles [6]

Below, the green buildings are on the left child and the purple buildings are on the right child. Note that the tiles overlap so the two green and one purple building in the center are not split.

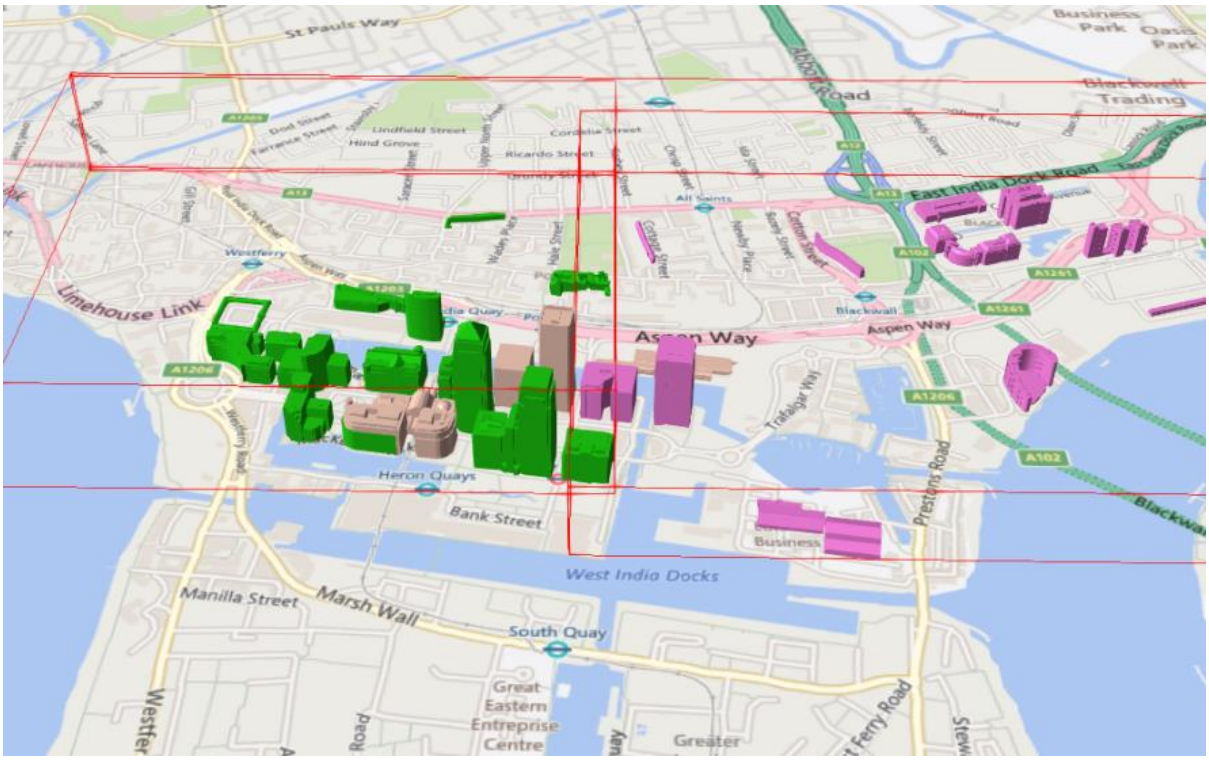

Figure 1.16 Loose quadtree example [6]

1.5.2. K-d Trees Tiling

A k-d tree is created when each tile has two children separated by a *splitting plane* parallel to the *x*, *y*, or *z* axis (or latitude, longitude, height). The split axis is often round-robin rotated as levels increase down the tree, and the splitting plane may be selected using the median split, surface area heuristics, or other approaches.

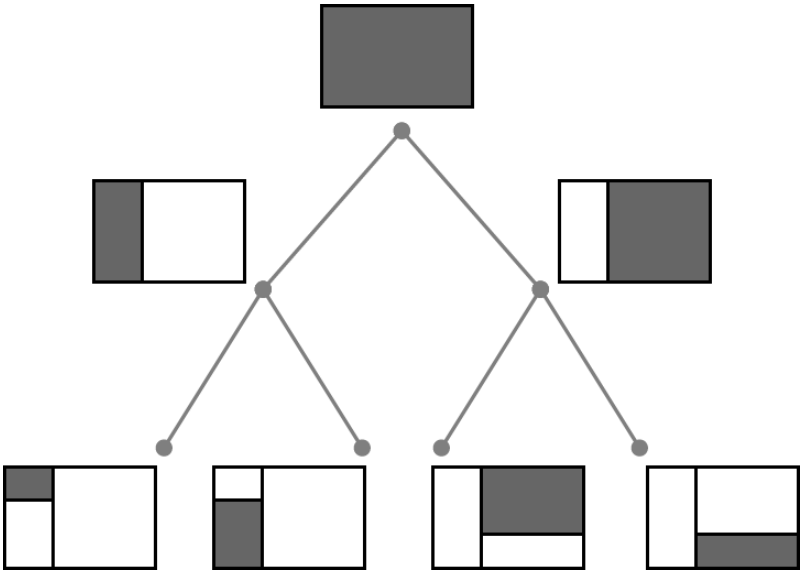

Figure 1.17 K-d trees [6]

Note that a k-d tree does not have uniform subdivision like typical 2D geospatial tiling schemes and, therefore, can create a more balanced tree for sparse and non-uniformly distributed datasets.

1.5.3. Octrees Tiling

An octree extends a quadtree by using three orthogonal splitting planes to subdivide a tile into eight children. Like quadtrees, 3D Tiles allow variations to octrees such as non-uniform subdivision, tight bounding volumes, and overlapping children.

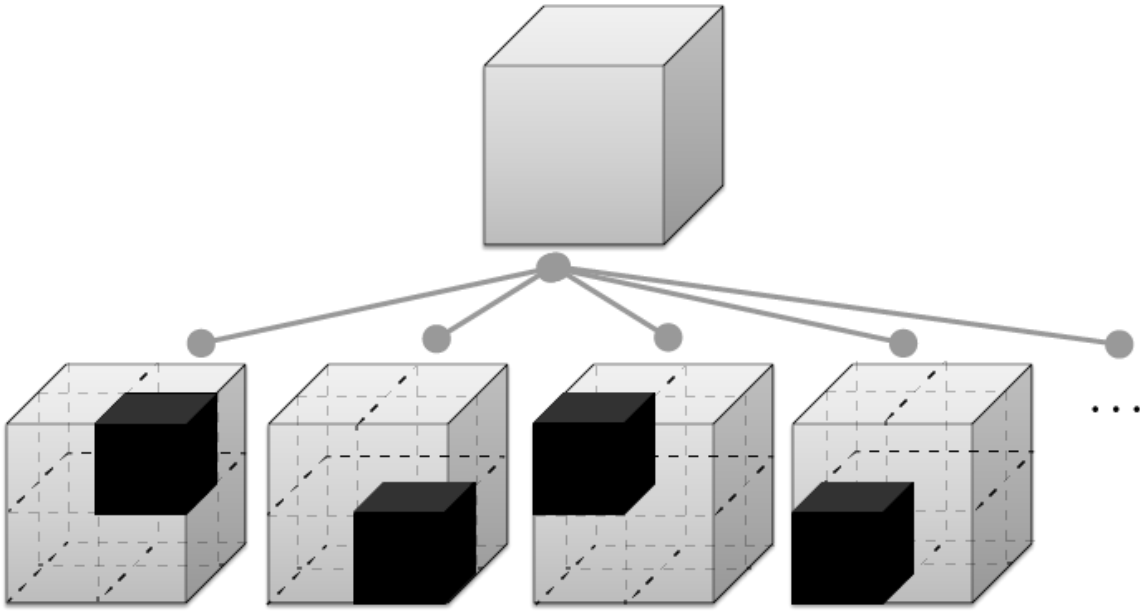

Figure 1.18 Traditional octree subdivision [6]

And in this thesis, this Tiling method will be employed as the primary approach for developing innovative techniques of partitioning.

CHAPTER 2. PROPOSED PARTITIONING METHODS

In the previous chapter, we have gone through some fundamentals about 3D Tiles, embedded file formats and some tiling methods that the community has developed. In this chapter, we are going to delve into our own Partitioning (Tiling) Method development.

2.1. Overview

2.1.1. Objectives

The primary objective of this thesis is to develop Partitioning Methods, which are capable of converting large glTF files into JSON tileset that can be efficiently streamed and visualized on WebGL platforms, in order to increase the rendering speed of traditional method. From that, we can improve the user experiences when using various 3D Tiles' applications.

In our research and development efforts, we concentrate on dividing the dataset based on a crucial factor - the maximum number of triangles that a single tile can accommodate.

2.1.2. Specifications

- Input: glTF file
- Output: JSON tileset
- Language: JavaScript
- Max number of triangles in a tile: 100 – 3500 triangles.

2.1.3. Partitioning Process Block Diagram

The project consists of three primary stages. The first stage involves obtaining the data from the glTF file. As mentioned, the glTF file contains vertex data, such as position, normal, and color. To achieve our goal of dividing the entire scene, it is essential to gather the complete dataset from the glTF file.

In the second stage, we proceed with the partitioning process, where we introduce the tiling algorithm. This algorithm is designed based on the vertex data obtained in the previous stage. Its purpose is to effectively divide the subject into tiles, optimizing the overall partitioning.

Finally, in the last step, we transform the partitioned subject into a tileset, effectively organizing and structuring the data for further use and applications.

The following diagram presents an overview of the entire system, illustrating the process of obtaining vertex information from the glTF file and subsequently converting it into a tileset. In the upcoming sections, a more in-depth exploration of each stage will be provided.

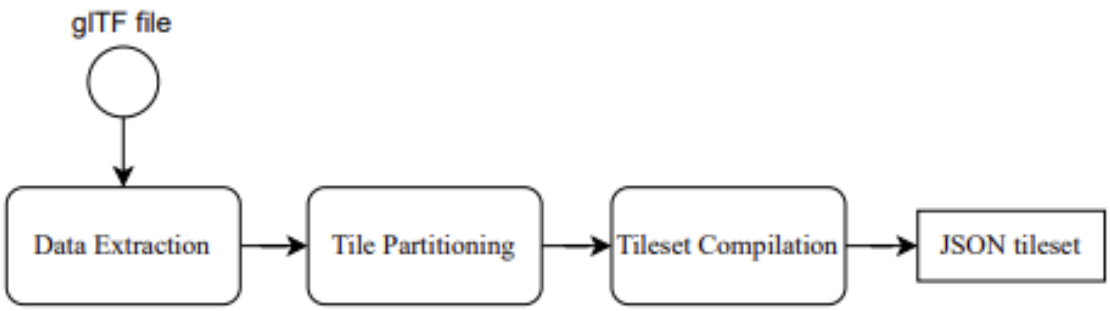

Figure 2.1 Partitioning Process Block Diagram

2.1.4. Sample Application Diagrams

This Section will illustrate a sample application bock diagram that the JSON tileset are used for rendering data. Client can send request to server in order to stream an glTF file. In return, the server will run the partitioning process based on the received information from the client: glTF data, partitioning method and value of maximum number of triangles per tile and then, the server will extract a JSON tileset and store it in the server.

The server now returns a state after successfully creating the tileset to the client. After receiving the state, the client of actor will automatically be directed to a rendering page which is running by Cesium JS platform and send requests to load each tile of the tileset from the server.

Moreover, actor can zoom, span or rotate the scene with their mouse. The figures below illustrate the Use-case Diagram and the Sequence Diagram of the application.

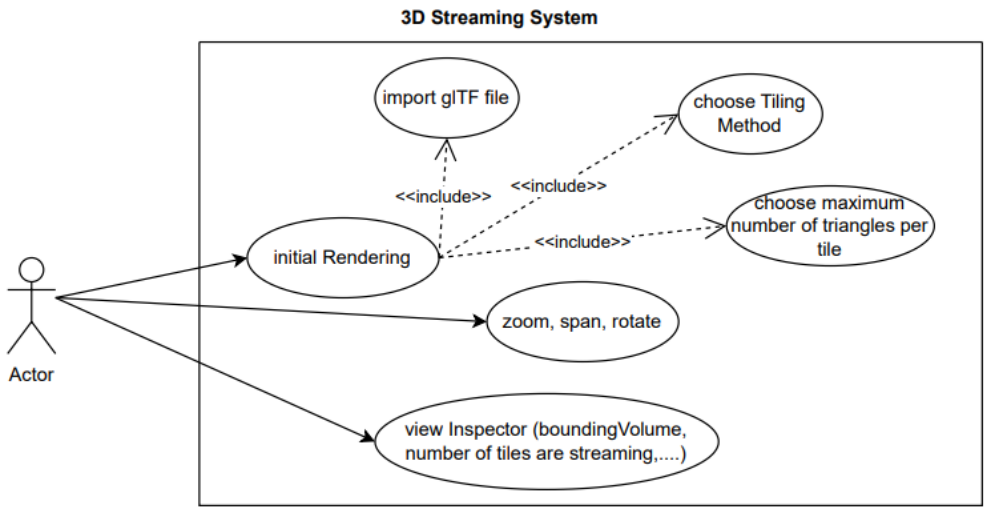

Figure 2.2 Use-Case Diagram

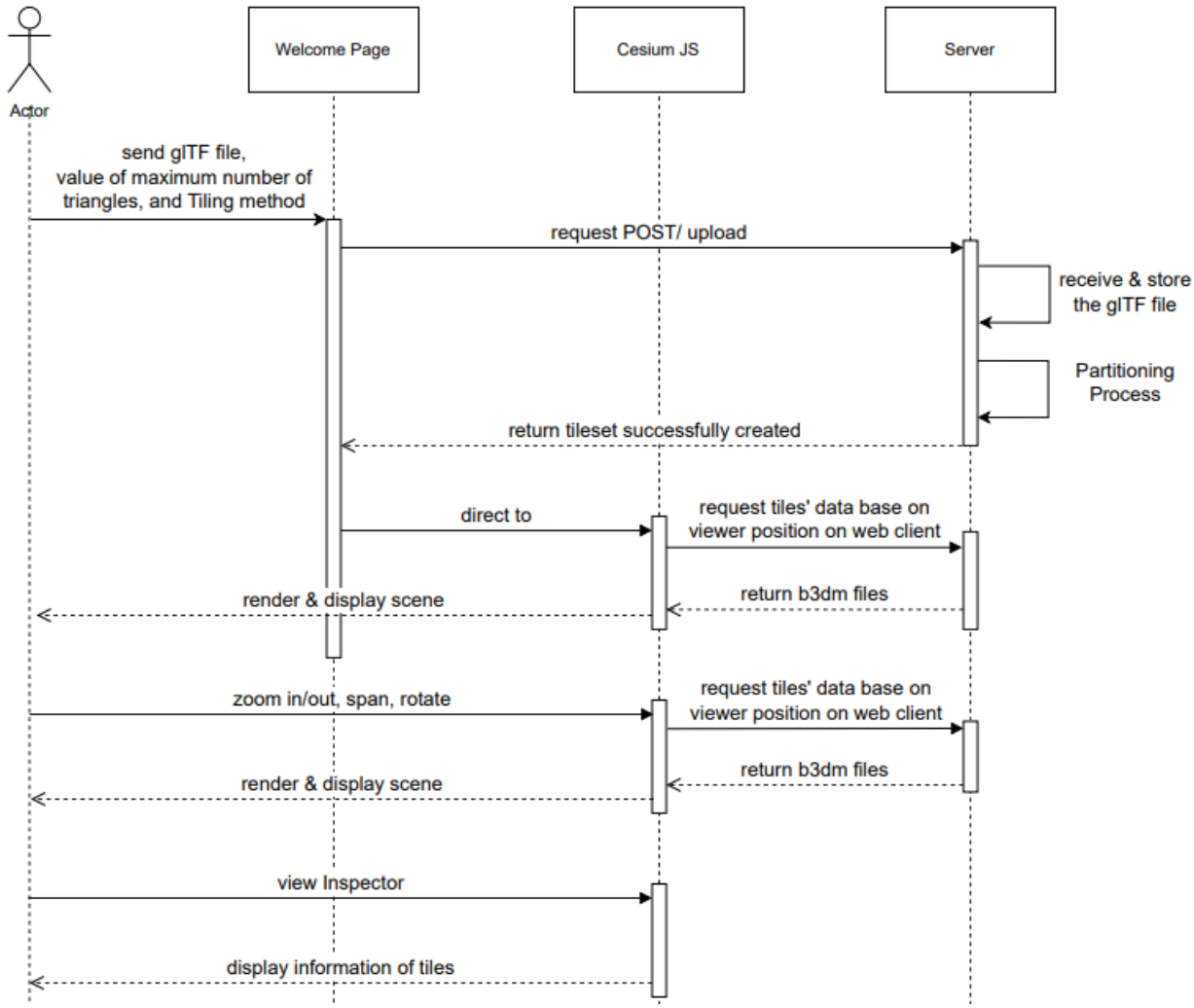

Figure 2.3 Sample Application Sequence Diagram

2.2. Proposed Methods

After careful consideration, we have opted to adopt the Octree Tiling method as our primary approach for partitioning. The decision stems from the specific requirements of our method, which revolves around effectively handling the maximum number of triangles within a single tile.

While the Quadtree Tiling technique divides the volume into four sections and further subdivides along two of the three axes, it may encounter certain scenarios where a tile would be tiled repeatedly to infinitely. This occurs when a substantial number of triangles share the same location on two axes but differ significantly on the third axis.

By implementing the Octree Tiling method, we address this limitation as it efficiently partitions the 3D space into eight octants, ensuring a more balanced distribution of triangles within each tile. This approach adapts dynamically to the complexity and density of the data, effectively managing the varying levels of detail and avoiding scenarios of infinite tiling.

2.2.1. Static Octree Tiling

This tiling method is primarily based on the principles of Traditional Octree Subdivision (see Section 1.5.3). Similar to the traditional approach, the root bounding volume is divided into eight parts by equally partitioning each axis (x, y, z) into two segments.

However, the key distinction lies in the consideration of the number of triangles within each tile. After the initial partitioning, the algorithm proceeds to traverse through each tile, checking the number of triangles it contains. If the condition of having an equal number of triangles in each tile is met, the algorithm moves on to the next tile. On the other hand, if the condition is not satisfied, the specific tile is further subdivided into eight smaller tiles.

An essential aspect of this method is the handling of tiles with zero triangles. To optimize the data structure, any tile found to have zero triangles is removed from the tile list.

The iterative process continues until all tiles meet the requirement of having an equal number of triangles.

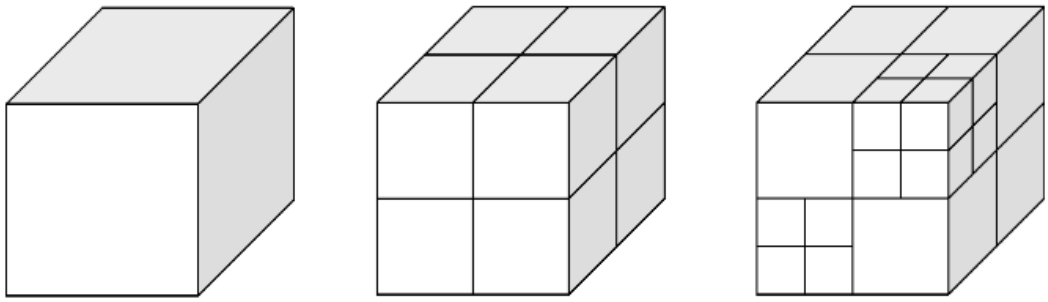

Figure 2.4 Statistic Octree Tiling [10]

2.2.2. Adaptive Octree Tiling

This tiling method draws inspiration from both Traditional Octree Subdivision and K-d trees (Section 1.5.2). Similar to Traditional Octree Subdivision, it divides the root bounding volume into eight smaller tiles. However, unlike the Static approach, where the center point of the bounding volume is always chosen for partitioning, this method employs an adaptive strategy.

The main concept is to determine a partition point in 3D space where the number of triangles on each side, corresponding to the x, y, and z axes, are equal. This partitioning ensures a balanced distribution of triangles, creating a symmetrical division of the 3D data.

After obtaining the list of triangle indices, we perform three separate ascending sorts based on the three different axes (x, y, z). Once sorted, we choose the middle

element from each sorted list to determine the partition point. These x, y, and z coordinates from the middle elements serve as the partition point for dividing the volume.

This iterative process continues until all tiles have an acceptable number of triangles, meeting the specified requirement. Additionally, if any tile is found to have zero triangles, we remove it from the list to optimize the data structure.

By employing this approach, we create an adaptive tiling method that achieves a balanced distribution of triangles within each tile, resulting in an efficient representation of the 3D scene.

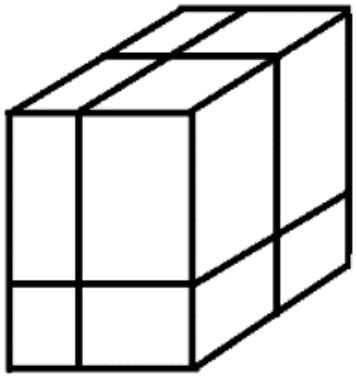

Figure 2.5 Adaptive Octree Tiling

2.3. Making Dataset For Tiling

2.3.1. OpenStreetMap

OpenStreetMap is a collaborative and open-source mapping project. It is a global community-driven initiative that allows individuals from around the world to contribute and edit geographic information, including roads, trails, buildings, rivers, and other points of interest.

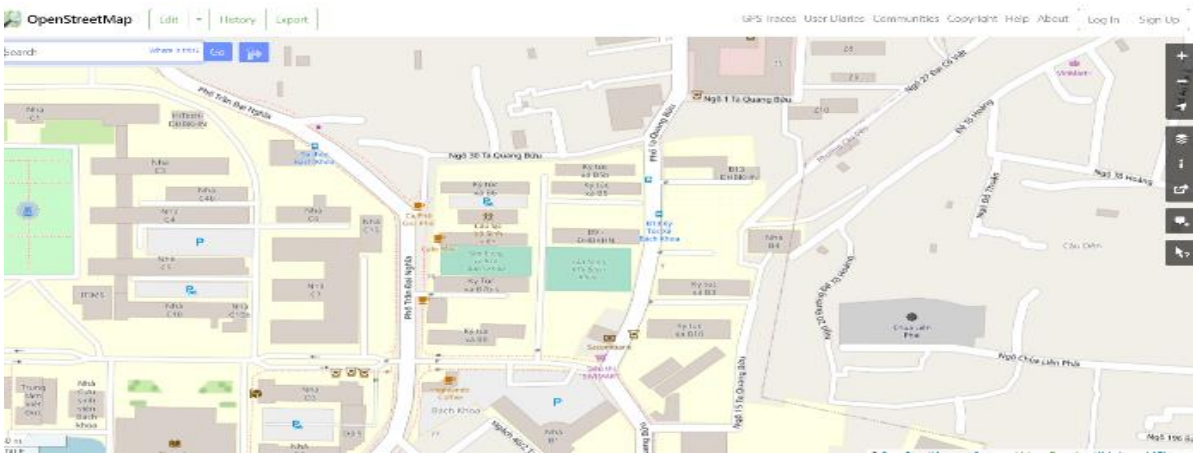

Figure 2.6 Screen of OSM on website [11]

In this application, the Export function allows users to access and download geographic data from OSM for specific regions or features of interest. We leverage this Export feature in OSM, which provides us with a downloadable file in the *.osm* format. This file contains all the relevant geographic data we require for further analysis and visualization in the subsequent stages of our research.

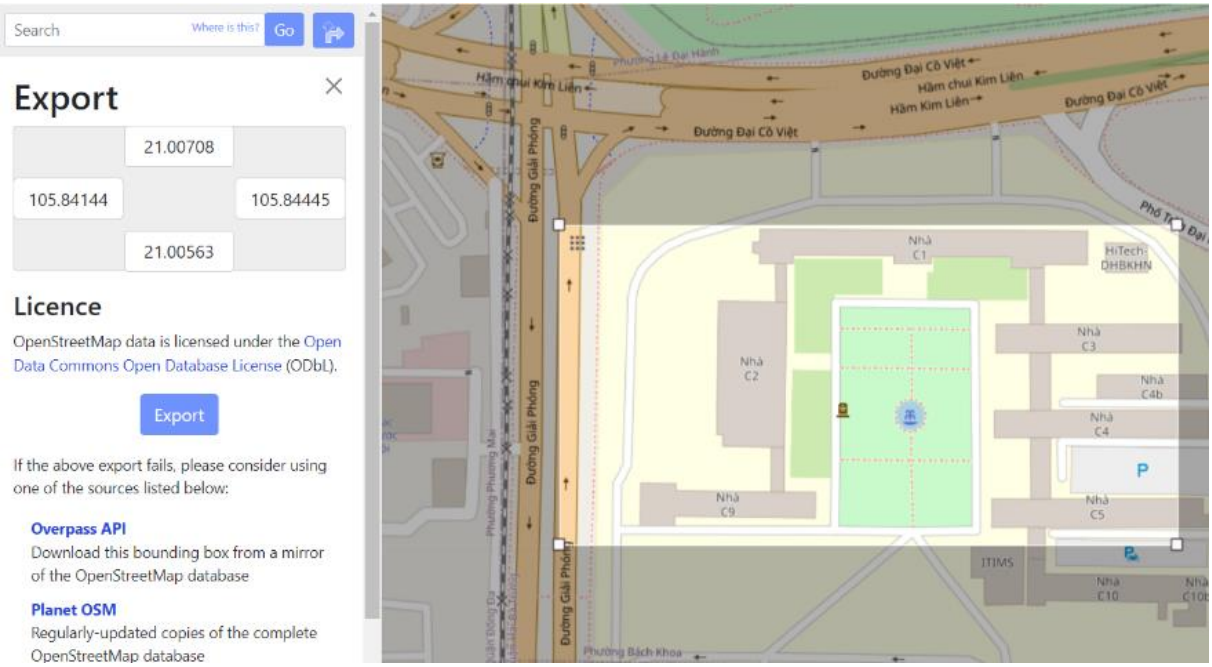

Figure 2.7 Screen of Export in OSM [11]

2.3.2. OSM2World

OSM2World is a converter that creates three-dimensional models of the world from OpenStreetMap data. It can be used as a stand-alone tool, on a server or as a library in Java programs.

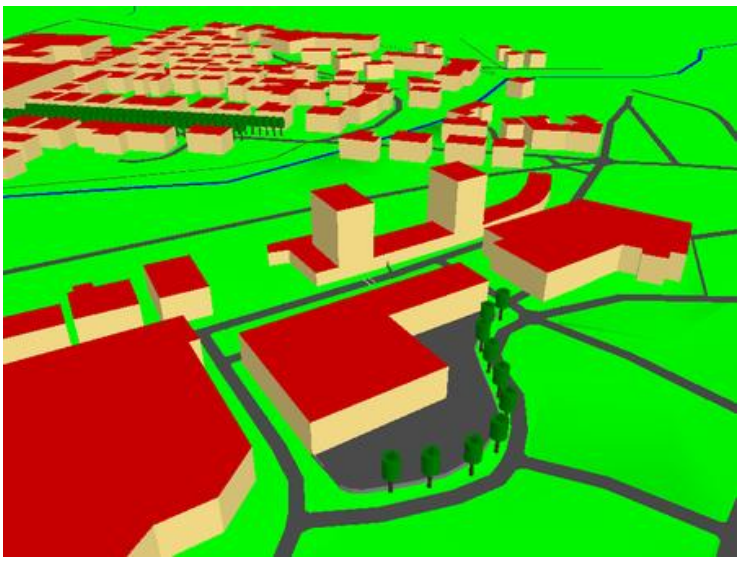

Figure 2.8 Example of .osm file being rendered by OSM2World [12]

With the help of this tool, it becomes possible to generate a glTF file from the recently acquired .osm file obtained from OpenStreetMap (OSM). This conversion allows for efficient transmission, loading, and rendering of 3D scenes, empowering users to create immersive and interactive experiences based on the rich and freely accessible OSM data.

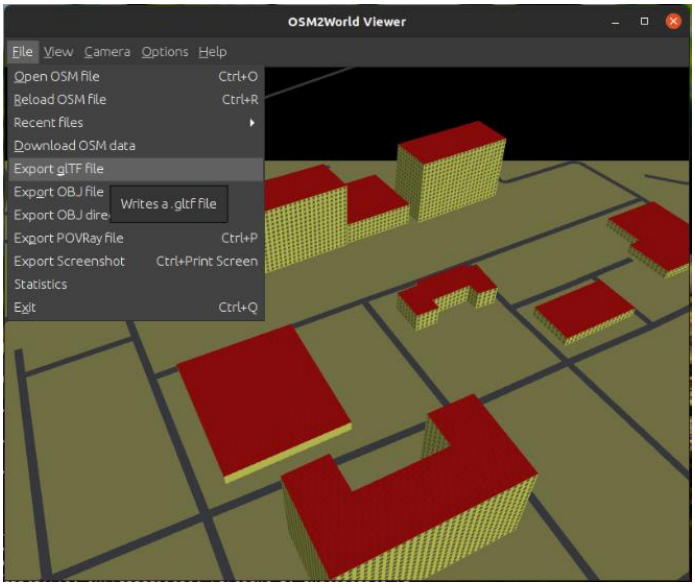

Figure 2.9 Some extracting function in OSM2World

2.4. Determining if a Triangle is in a Tile

In the process of partitioning the metadata, various challenges may arise, one of which involves deciding whether a particular triangle should remain in a specific tile. Additionally, when we divide the bounding volume into eight parts, there may be instances where triangles overlap.

To address these obstacles, we have adopted a solution that involves selecting the centroid of each triangle to determine its corresponding tile. This is achieved by taking the coordinates of the three vertices of the triangle and calculating their average, which serves as the central point for the triangle. This approach allows us to accurately assign each triangle to the appropriate tile while mitigating potential overlaps and ensuring an effective partitioning of the metadata.

For example, a triangle has 3 vertices, A (x1, y1, z1), B (x2, y2, z2), C (x3, y3, z3), so the centroid of the triangle will be I (a, b, c) where:

$$I(a,b,c) = \begin{cases} a = \dfrac{x1 + x2 + x3}{3} \\ b = \dfrac{y1 + y2 + y3}{3} \\ c = \dfrac{z1 + z2 + z3}{3} \end{cases} \qquad \text{Equation 2.1}$$

CHAPTER 3. ALGORITHM DEVELOPMENT

In this chapter, we will present the comprehensive implementation of the Partitioning Methods proposed in Chapter 2. We will run through each stage that represent in Figure 2.1, from how to extract data from the glTF file in order to partitioning (Section 3.1), to how choosing the dividing point makes Static Octree Tiling different from Adaptive Octree Tiling. And finally, is the complication of tiles to create a final JSON tileset.

3.1. Data Extraction

The process initiates by retrieving the glTF (GL Transmission Format) file, a widely used format for 3D models and scenes as mentioned in Chapter 1. The data extraction stage involves parsing the glTF file to access the necessary vertex information. This includes coordinates, normals, textures, and other relevant attributes, which together constitute the foundation of the 3D model.

Below is the block diagram explain the details process of this stage.

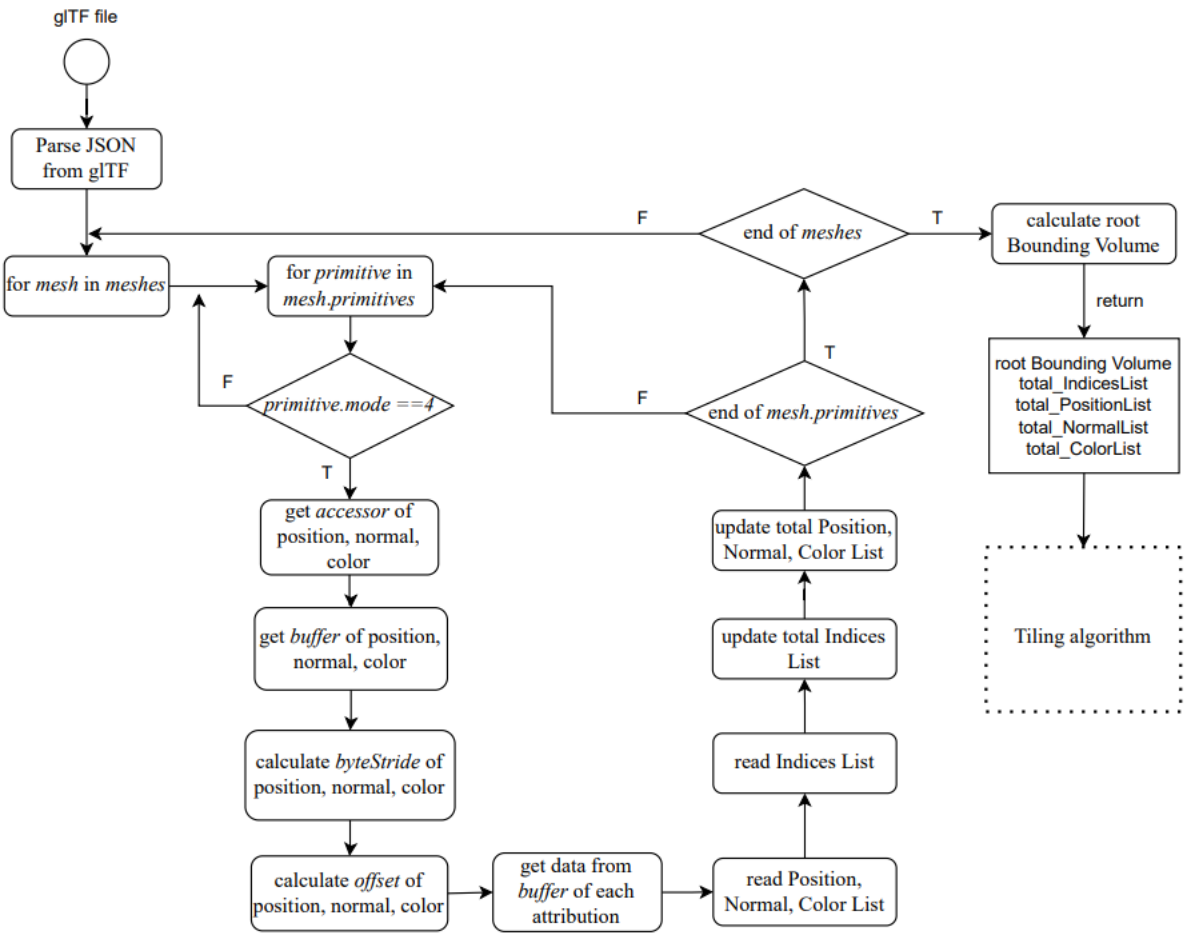

Figure 3.1 Block Diagram Of Data Extraction Stage

As you can see from the diagram, the main goal of this process is to get the data of position, normal, color of each mesh and the indices list in the scene.

3.2.1. Parsing glTF

In Section 1.2, we have gone through the structure of glTF file, which is a node hierarchy JSON file with the object hierarchy as shown in Figure 1.9. The first thing we need to do in this process is to parse the information of the given glTF file and extract its data for further processing.

```
function ParseGLTF(gltfPath) {
  const gltfData = fs.readFileSync(gltfPath);
  const gltf = JSON.parse(gltfData);
  return gltf;
}

const gltf = readData.ParseGLTF(gltfPath);
//readData is the library we store the ParseGLTF function
```

gltfPath in the code above is the path points to the location of the glTF file that you want to extract. We use *readFileSync* to read the entire contents of a file and return it as raw binary data.

After reading the glTF file and obtaining it raw binary data, the *JSON.parse* function is used to parse that binary data into a JavaScript object. The *JSON.parse* function converts a JSON-formatted string or raw binary data (in this case, *gltfData*) into a JavaScript object representation. The resulting *gltf* variable holds the parsed glTF data as a JavaScript object.

3.2.2. Accessing Vertex Information

The goal of this part is to get the full list of vertex position, normal, color and the corresponding indices list of it. Each one of this attributes will be declared as array type in the code.

```
let positionList = [];
let normalList = [];
let indicesList = [];
let colorList = [];
```

3.2.2.1 Get Accessors

In the glTF file that we extract using the OSM2World tool (Section 2.3.2), there is no specific pointer for the indices array. However, the position, normal and color are clearly stored in the element of the *meshes*.

```
"meshes": [
    {
      "primitives": [
        {
          "attributes": {
            "COLOR_0": 2,
            "POSITION": 0,
            "NORMAL": 1
          },
          "material": 0,
          "mode": 4
        }
      ]
    },
    {
      "primitives": [
        {
          "attributes": {
            "COLOR_0": 6,
            "POSITION": 3,
            "TEXCOORD_0": 5,
            "NORMAL": 4
          },
          "material": 1,
          "mode": 4
        }
      ]
    }
   ]
```

The above JSON code that got from the original glTF file, which is extracted by OSM2World, *meshes* represents a 3D object which is an array of one or many *primitives*. A primitive is the basic building block of mesh in glTF. It defines the geometry and appearance of a specific part or element of the 3D object. Each primitive corresponds to a specific set of attributes like position, normal, color, texture coordinates, etc.

In the provided code snippet, each attribute is associated with its own unique index, which serves as a reference to the corresponding *accessor* containing the actual attribute data. This indexing mechanism allows us to access and process the attribute information effectively. By retrieving the data from the specified accessors using these indexes, we can obtain the necessary geometry details such as vertex positions, normals, colors, and texture coordinates. This approach ensures efficient data organization and enables seamless data sharing across different parts of the 3D model represented in the glTF file.

```
for (mesh of gltf.meshes) {
        for (primitive of mesh.primitives) {
            if (primitive.mode === 4) {/*get accessor,...*/}
```

We create a loop to traverse through every object in the *meshes* array and called each one of them *mesh*. For each *mesh*, we traverse through each element of its

primitives and called it *primitive*. This *primitive* is where we get the index that points to accessor of attributes like position, normal, etc.

One big thing to be noticed is the *mode* of *primitive* should be equal to 4, since it corresponds to a rendering mode called GL_TRIANGLES. In this rendering mode, the vertices of the primitive are organized into groups of three to form individual triangles. Each triplet of vertices represents one triangle, and the graphics engine renders these triangles to create the visible surface of the 3D object.

```
//get Accessor
let positionAccessor = gltf.accessors[primitive.attributes.POSITION];
let normalAccessor = gltf.accessors[primitive.attributes.NORMAL];
let colorAccessor = gltf.accessors[primitive.attributes.COLOR_0];
```

The code above uses the accessor index that obtained from the attribute keys to retrieve the *accessor* that holds the vertex data for the *primitive*.

After getting the "storage" of each attribute, we jump into their *accessors* for further information.

```
"accessors": [
      {
            "bufferView": 0,
            "componentType": 5126,
            "count": 36,
            "type": "VEC3",
            "max": [-197.542, 3.0, 123.741],
            "min": [-197.942, 2.3, 123.721]
      },
      {
            "bufferView": 1,
            "componentType": 5126,
            "count": 36,
            "type": "VEC3",
            "max": [1.0, 1.0, 1.0],
            "min": [-1.0, -1.0, -1.0]
      }
    ]
```

Note that *accessors* is also an array, and the index that we got from the previous step is the position of its attribute in this array. As you can see from the JSON sample of *accessors* above, each element contains *bufferView*, *componentType*, *count*, *type*, *max*, *min*. All of these elements we have explained in Chapter 1, and the following step in this process is to get the *buffer*, *byteStride*, *offset* of each position, normal and color attributes.

3.2.2.2 Get Buffers

Buffer is property refers to the index of the buffer that contains the binary data for the attribute. To get *buffer*, we need to get the value of *bufferView* in the *accessor* first, which is the index points to the position in the *bufferViews* array.

```
"bufferViews": [
      {
            "buffer": 0,
            "byteLength": 16
      },
      {
            "buffer": 1,
            "byteLength": 16
      }
     ],
"buffers": [
      {
            "uri": "data:application/gltf-buffer; base64,J/FFwzMzE0”,
            "byteLength": 16
      },
      {
            "uri": "data:application/gltf-buffer; base64,J/FFwzMzE0”,
            "byteLength": 16
      }
     ]
```

In this *bufferViews*, each element has their own *buffer* property, and it will have the index to access the *buffers* array, where contain the *buffer* of each attribute. For example, in the code above, the first *buffer* of *bufferViews* is 0, which points to the first element in the *buffer* array.

The code below shows in detail how we implement it.

```
//get BufferView
let positionBufferView = gltf.bufferViews[positionAccessor.bufferView];
let normalBufferView = gltf.bufferViews[normalAccessor.bufferView];
let colorBufferView = gltf.bufferViews[colorAccessor.bufferView];

//get Buffer
let positionBuffer = gltf.buffers[positionBufferView.buffer];
let normalBuffer = gltf.buffers[normalBufferView.buffer];
let colorBuffer = gltf.buffers[colorBufferView.buffer];
```

3.2.2.3 Get ByteStrides

ByteStride as explained specifies the number of bytes between consecutive elements (e.g., vertices) in the attribute data. It is used to determine the distance between each element in binary data. This property is particularly relevant when dealing with interleaved data where multiple attributes are packed into the same buffer.

Each attribute has its own *byteStride*, so it is important to implement them distinctively. However, they use the same method to calculate their *byteStride*. Based on section 1.3.7.2 , the Table 1.1 and Table 1.2, we have a mathematic function below:

byteStride = bytes of *componentType* * number of components in *type*

componentType and *type* we get from the above *accessor*, and we have to separate each case of them to get the desired value for the *byteStride*.

```
//calculate bytestride
let positionByteStride = calculate.ByteStride(positionAccessor);
let normalByteStride = calculate.ByteStride(normalAccessor);
let colorByteStride = calculate.ByteStride(colorAccessor);

//these function is stored in calculate library
function ByteStride(Accessor) {
    let byteStride = 12;
    const componentTypeSize =getComponentTypeSize(Accessor.componentType);
    if (Accessor.type === "VEC3") {
        byteStride = componentTypeSize * 3;
    } else if (Accessor.type === "VEC2") {
        byteStride = componentTypeSize * 2;
    } else if (Accessor.type === "SCALAR") {
        byteStride = componentTypeSize * 1;
    } else {
        return;
    }
    return byteStride;
}

function getComponentTypeSize(componentType) {
    switch (componentType) {
        case 5120: // BYTE
        case 5121: // UNSIGNED_BYTE
            return 1;
        case 5122: // SHORT
        case 5123: // UNSIGNED_SHORT
            return 2;
        case 5125: // UNSIGNED_INT
        case 5126: // FLOAT
            return 4;
        default:
            return 0;
    }
}
```

3.2.2.4 Get Offsets

The *offset* property indicates the byte offset within the buffer where the attribute data starts. It defines the starting point from which the attribute data is read in the buffer. By specifying the *offset*, we can access the correct section of the buffer containing the data for a specific attribute.

Offset is calculated by the formula below:

$$\text{offset} = \text{accessor.byteOffset} + \text{bufferView.byteOffset} \qquad \text{Equation 3.1}$$

3.2.2.5 Read Data

Next, we need to read the data from the *buffer* above. In each *buffer*, there is an *uri* that contains the data to be read. We need to create a function that takes this *uri* as an input and returns the binary data extracted from the given *uri*. It should support two circumstances: the *uri* contains base64 encoded data and the *uri* points to a binary file with *.bin* file extension.

```
//get Data
let positionData = readData.parseDataFromURI(positionBuffer.uri);
let normalData = readData.parseDataFromURI(normalBuffer.uri);
let colorData = readData.parseDataFromURI(colorBuffer.uri);
```

If the *uri* is base64 encoded data, which would start with "data:application/gltf-buffer;base64", we decode the base64 data into raw binary data using function Buffer.from and return it as below:

```
const dataPrefix = "base64,";
const base64Data = uri.substring(
                          uri.indexOf(dataPrefix) + dataPrefix.length,);
const decodedData = Buffer.from(base64Data, "base64");
return decodedData;
```

If the *uri* is a binary file, we should check if the *uri* ends with ".bin", then use the function below to read the binary data from the file, the raw binary data is converted into an object which contains the binary data to return.

```
const rawData = fs.readFileSync(uri);
const typedArray = Buffer.from(rawData);
return typedArray;
```

3.2.2.6 Read & Update Each Attribute List

This is the last step in the Parse glTF stage. After having the buffer data, the offset, the byteStride of each attribute, we continue to create the function to read and update the data of them.

Below is the procedure we used to read the list of each attribute of each *mesh*:

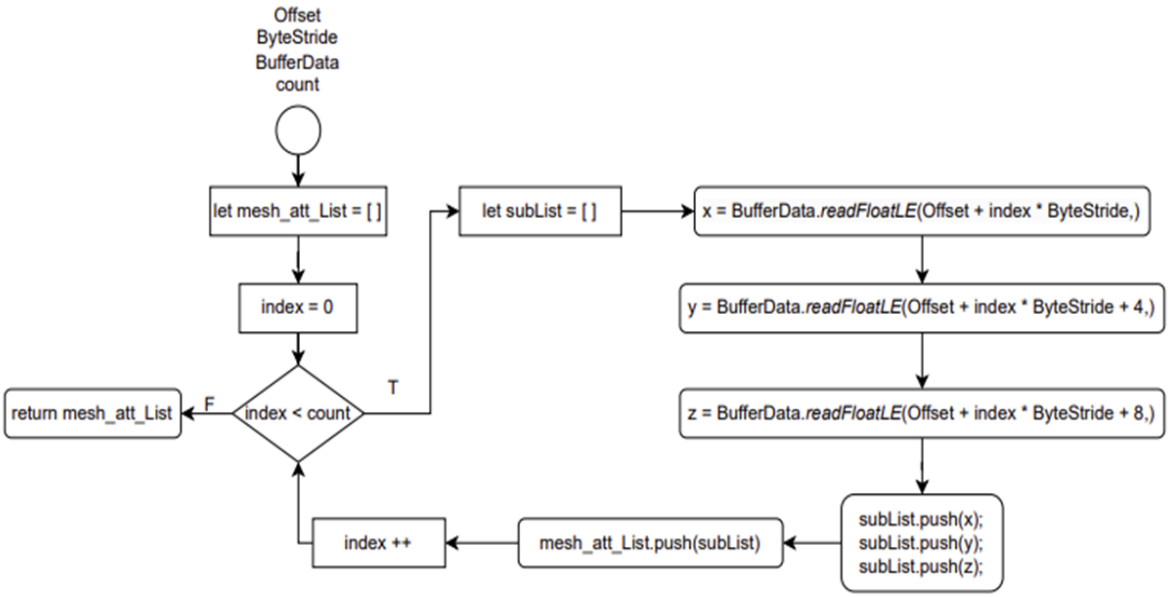

Figure 3.2 Read Data Process for each Attribute

We have the *offset*, *byteStride*, *data* and *count* as the main parameter of this process. Let take the *position* for example, the *count* is taken from the position's accessor, define the number of vertex count in the position data.

The function initializes an empty array called *mesh_position_List*, which will hold the extracted vertex positions. It then iterates over each vertex in the position data, using the *count* parameters as the upper limit of the loop. For each vertex, it calculates the byte offset for reading its position data within the buffer.

The position data for each vertex is represented as a 3D vector (x, y, z). The function reads the x, y and z components of the vertex position using the *readFloatLE(offset)* function, where offset is the calculated byte offset for the current vertex. This *readFloatLE* function reads floating-point values in little-endian format from the binary buffer. The function assumes that the position data is stored as three consecutive 32-bit floating-point numbers (x, y, z), each occupying 4 bytes. The byte offsets are calculated based on this assumption.

After reading the x, y and z components, it constructs a 3D array *subList* containing the vertex position data (x, y, z). Finally, it adds *subList* to the *mesh_position_List*, effectively storing the vertex position data for all vertices.

This function is being used in the same way for normal and color lists. However, the return value of this function is only the list of this *mesh*. What we want to extract is the total list of the whole scene, that's why after conducting this function, we need to have one more step.

```
total_positionList = total_positionList.concat(mesh_position_List);
total_normalList = total_normalList.concat(mesh_normal_List);
total_colorList = total_colorList.concat(mesh_color_List);
```

The *total_positionList*, *total_normalList*, *total_colorList* are the public arrays we have declared at the beginning of this process, while the *mesh_position_List*, *mesh_normal_List*, *mesh_color_List* are the arrays that we have extracted using the function above.

The *concat* function is used to returns a new array containing the elements from all the input arrays in the order they appear in the parameter list. In this code, the *concat* function takes *mesh_position_List* as a parameter and updates the *total_positionList* with a new array that contains all the elements from both *total_positionList* and *mesh_position_List*, preserving their order. The original arrays remain unchanged, and *total_positionList* now contains the combined elements of both arrays.

One last thing to do is to create a list containing the indices information, which would help us to select the triangle in the Partitioning part.

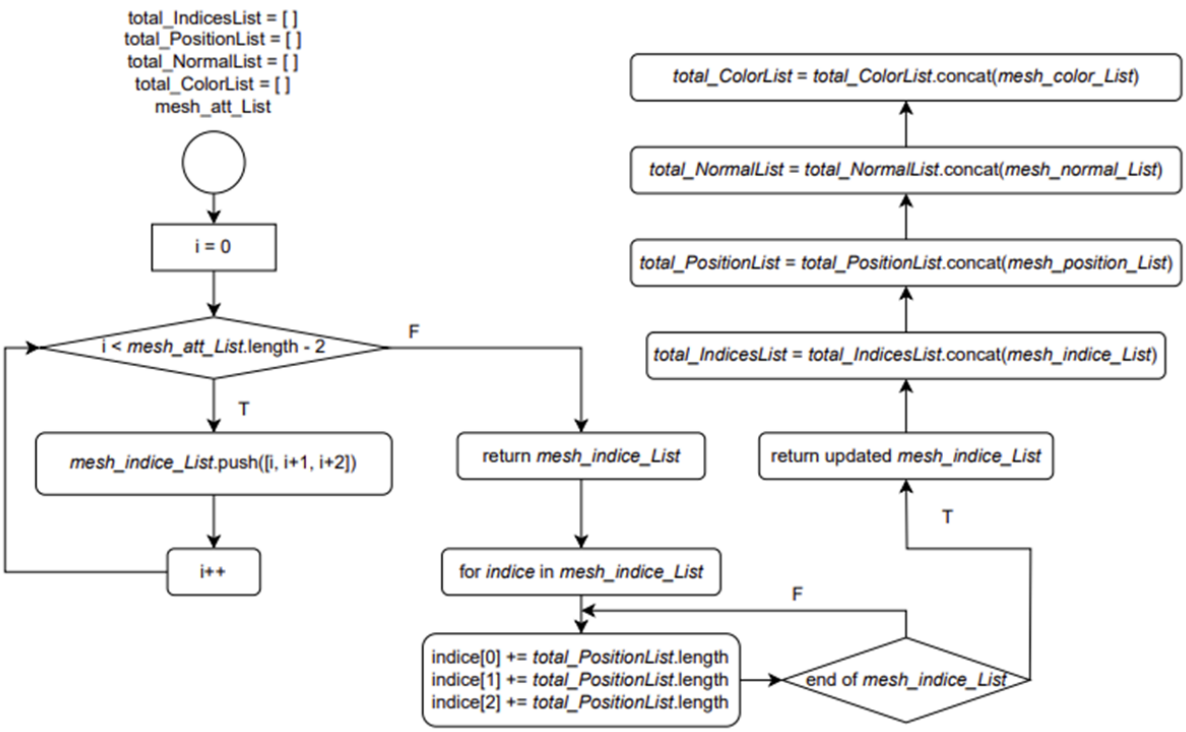

Figure 3.3 Update total Indices List and other Attribute's Lists

This function takes the length of the array *mesh_position_List* above as input and returns a new array *mesh_indice_List*. Imagine each 3 vertex position equals to one triangle, so we set *smallList* as a sub array with contains the order of each vertex, and then add each *smallList* to the total *mesh_indice_List*.

The result in return would be like this: [[0, 1, 2], [1, 2, 3], [2, 3, 4],...].

But using only this function is not enough to get the total indices list. This function returns the indices list of this *mesh*, not the list of the whole scene. We cannot use the public array *total_positionList* to take into this function because if we take it into account, there will be some excessive triangles in comparison to the original dataset. For example, the last vertex of *mesh 0* being grouped with the first and second vertex of *mesh 1* is not the result that we want.

Finally, we come up with an additional function which will update the indices list with each *mesh*.

```
function reIndexTotalIndices(
                indiceList, indicesListNoIndex, positionList) {
    for (let i = 0; i < indicesListNoIndex.length; i++) {
        let index = indicesListNoIndex[i];
        for (let j = 0; j < index.length; j++) {
            index[j] += positionList.length;
        }
    }

    indiceList = indiceList.concat(indicesListNoIndex);
    return indiceList;
}
```

This function takes the *indicesListNoIndex* and *positionList* as input and returns the updated public array *indiceList*. It will take the *indicesListNoIndex* and re-index it by adding the length of the current public *positionList*, so that the *indicesListNoIndex* will be separated with other indices list of other meshes, and also keep up with the length of the *positionList.*

It is important to note that the first element of each indices list for each mesh must be set equal to the current length of *positionList* before updating. This re-indexing process for *indicesList* is performed as a preliminary step before updating other attributes' lists.

3.2.2.7 Calculate Root Bounding Volume

The system continues until every *mesh* iterated, and the list of position, normal, color and indices are updated. Next, we need to calculate the root Bounding Volume of the big scene. The root Bounding Volume is a vital element for the next stage of tiling, which will need the bounding volume to divide each tile.

To get the root Bounding Volume, we once again traverse through every *mesh and primitive*, but only take the *accessor* of position, which represents for coordinate of vertices.

First, we declare the bounding volume with *minX*, *minY*, *minZ* equal to Infinity, and *maxX*, *maxY*, *maxZ* equal to negative Infinity. Then, through each *accessor* of position, there are *min* and *max* properties that define the bounding volume of that *accessor*. We can compare the current root Bounding Volume with it to find the largest bounding volume.

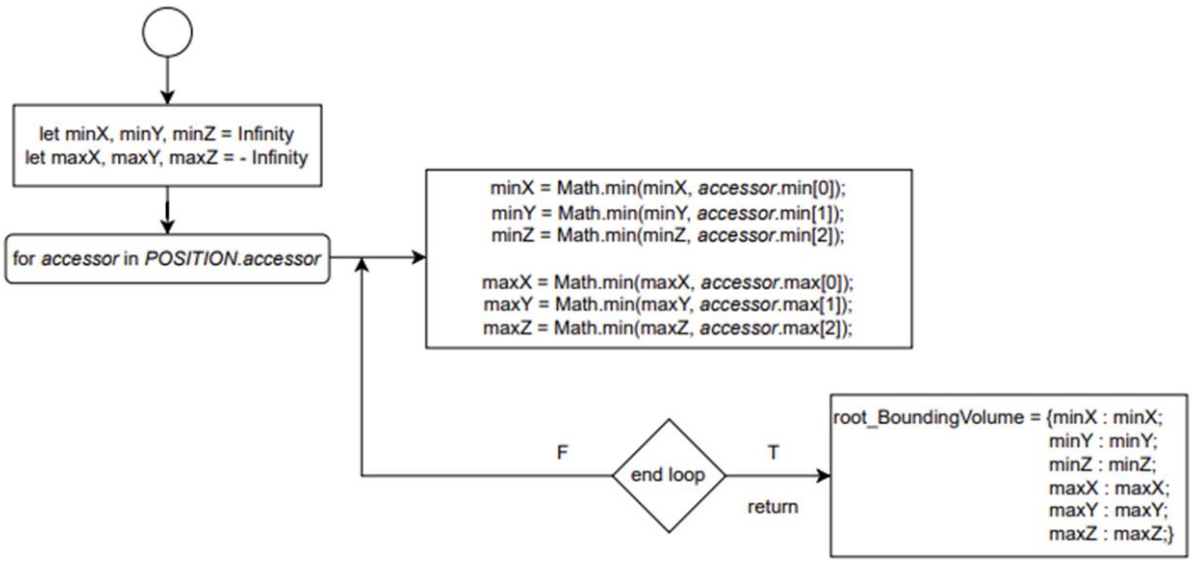

Figure 3.4 Calculate Root Bounding Volume

Each of *min* and *max* property has 3 element, corresponding to min or max of x, y, z. We use the Math.min function to compare the current root volume with the accessor volume to update the minimum part of root bounding volume, and in the maximum part we use Math.max function.

After a full traversal, the system will return the updated root Bounding Volume in terms of an object with min x, y, z and max x, y, z.

3.2. Tile Partitioning

To enhance comprehension of the tiling process, the following figure illustrates a detailed block diagram:

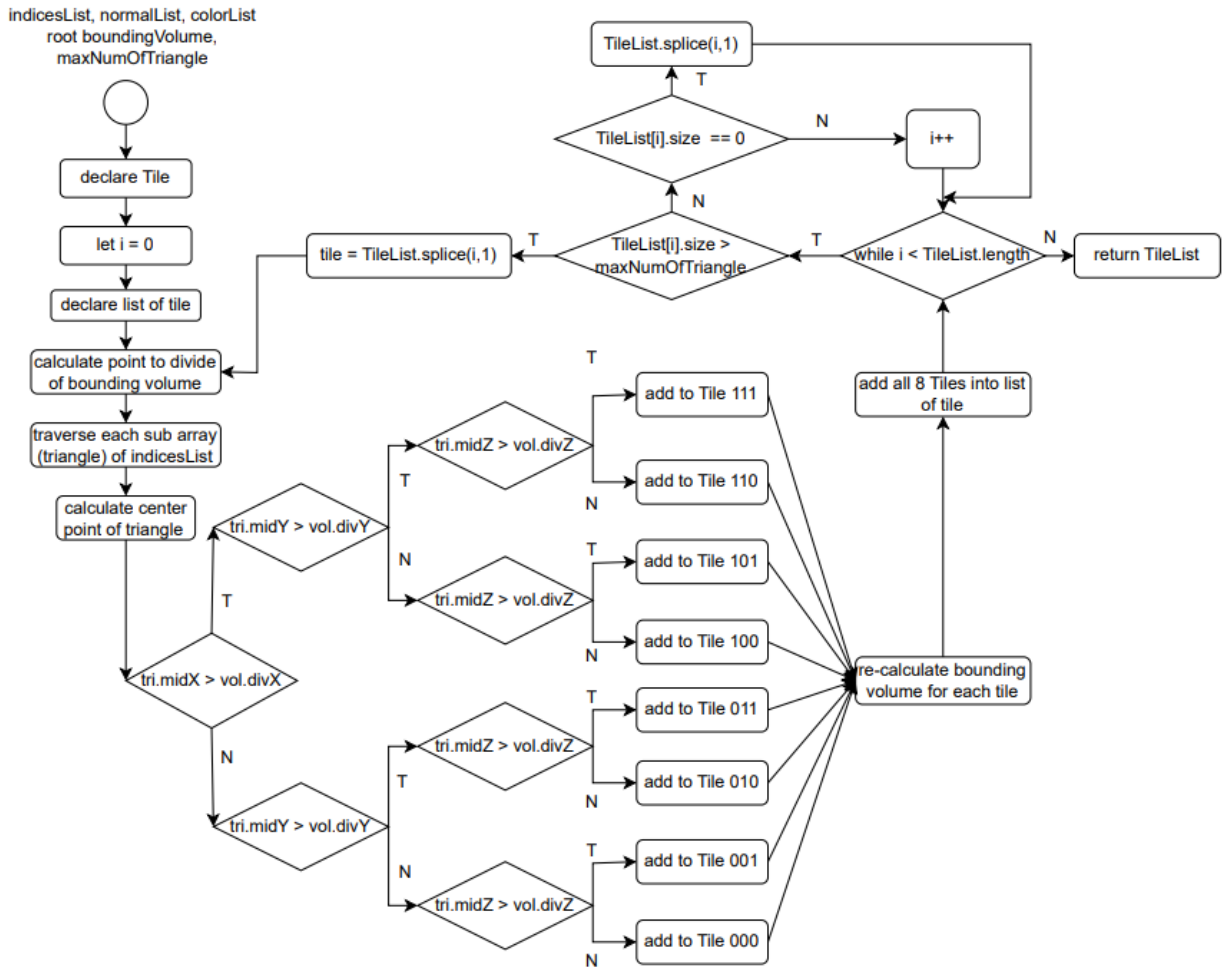

Figure 3.5 Block Diagram Of Tile Partitioning Stage

In this stage, we take the *indiceList*, *positionList*, *normalList*, *colorList* and the root Bounding Volume, which are calculated in the previous part, as the inputs of the Tiling process. And one thing is the most important in our methods, is the maximum number of triangles in each tile.

3.3.1 Declare Tile

To create a tileset, we have to divide the whole scene into smaller volumes with its corresponding data of vertices and color. Moreover, each tile at the end of the tiling process will be transformed into a glTF file and finally a b3dm file, which will be the main factor of storing data in a tileset.

For that reason, first we have to define a class for a Tile. This class should hold some characteristic that represents a sample Tile.

```
class Tile {
    constructor( level, xyz, indiceList, positionList, normalList,
        colorList, size, boundingVolume, uri,)
    {
        this.level = level;
        this.xyz = xyz;
        this.indiceList = indiceList;
        this.positionList = positionList;
        this.normalList = normalList;
        this.colorList = colorList;
        this.size = size;
        this.boundingVolume = boundingVolume;
        this.uri = uri;
    }
}
```

Each tile would have their own *indiceList*, *positionList*, *normalList*, *colorList* and their own *boundingVolume*. This is the main data of each tile, which will be encoded back to the base64 before being rendered in Cesium.

Level represents the level of partitioning, while the *xyz* defines the position of that tile in the scene. Both *level* and *xyz* are part of the *uri* and it will be used as the name for the transformation of files, which will be explained in Section 3.3. However, we can imagine the uses of *level* and *xyz* through the figure below.

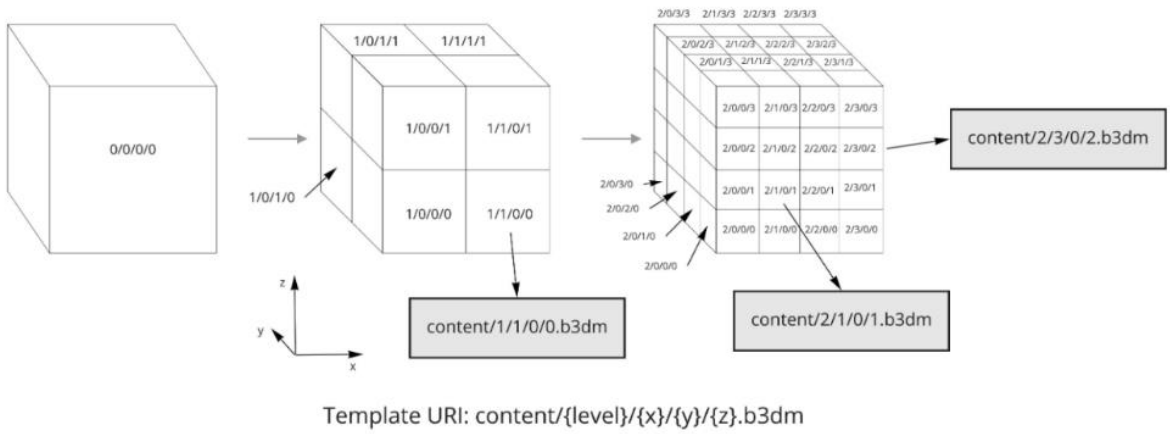

Figure 3.6 Example of level and xyz combination in URI [10]

The *size* is the number of triangles in a tile, and it is used for checking if a tile satisfies the maximum number of triangles in each tile.

3.3.2 Declare List of Tiles

TileList is an array that contains *Tile* type object. This array is created for storing the tiles after each partitioning level. Additionally, in the coming steps, the list will be used for checking the condition of maximum number of triangles in a tile, which is the most important in this project.

3.3.3 Static Octree Tiling

To divide the scene into smaller scenes, as discussed in Chapter 2, we choose the Octree Tiling as the main method. In order to accomplish that, we need a point as a dividing point in the (root) bounding volume.

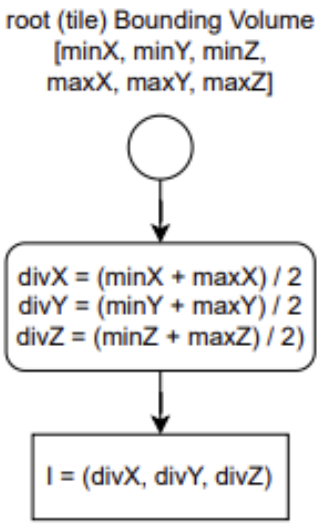

Figure 3.7 Choosing Dividing Point in Static Octree Tiling

At first, we have the (root) bounding volume in the form of: {minX, minY, minZ, maxX, maxY, maxZ}.

As this is static tiling, which is divide equally the bounding volume into 8 parts, hence we can calculate the dividing point I(divX, divY, divZ) like the following:

$$I(divX, divY, divZ) = \begin{cases} divX = \frac{minX+maxX}{2} \\ divY = \frac{minY+maxY}{2} \\ divZ = \frac{minZ+maxZ}{2} \end{cases} \qquad \text{Equation 3.2}$$

3.3.4 Adaptive Octree Tiling

In the method of Adaptive Tiling, we introduce a process for determining the dividing point, denoted as I(divX, divY, divZ). Unlike the Static Tiling approach, this method involves dividing the bounding volume based on the distribution of triangles within it. The goal is to identify a point where there exists a symmetrical distribution of triangles along each axis.

To achieve this, we select a specific point I(divX, divY, divZ), where the number of triangles with x-coordinates smaller than divX is equal to the number of triangles with x-coordinates greater than divX. The same principle applies to the y and z coordinates as well. In other words, the number of triangles on one side of the point is mirrored on the other side along each axis, creating a balanced distribution of triangles.

So, to get the point as we wanted, we need to sort out the list of triangles. The idea is first to get the list that contains the centroid of each triangle. We use the *indicesList* and *positionList* to calculate these centroids.

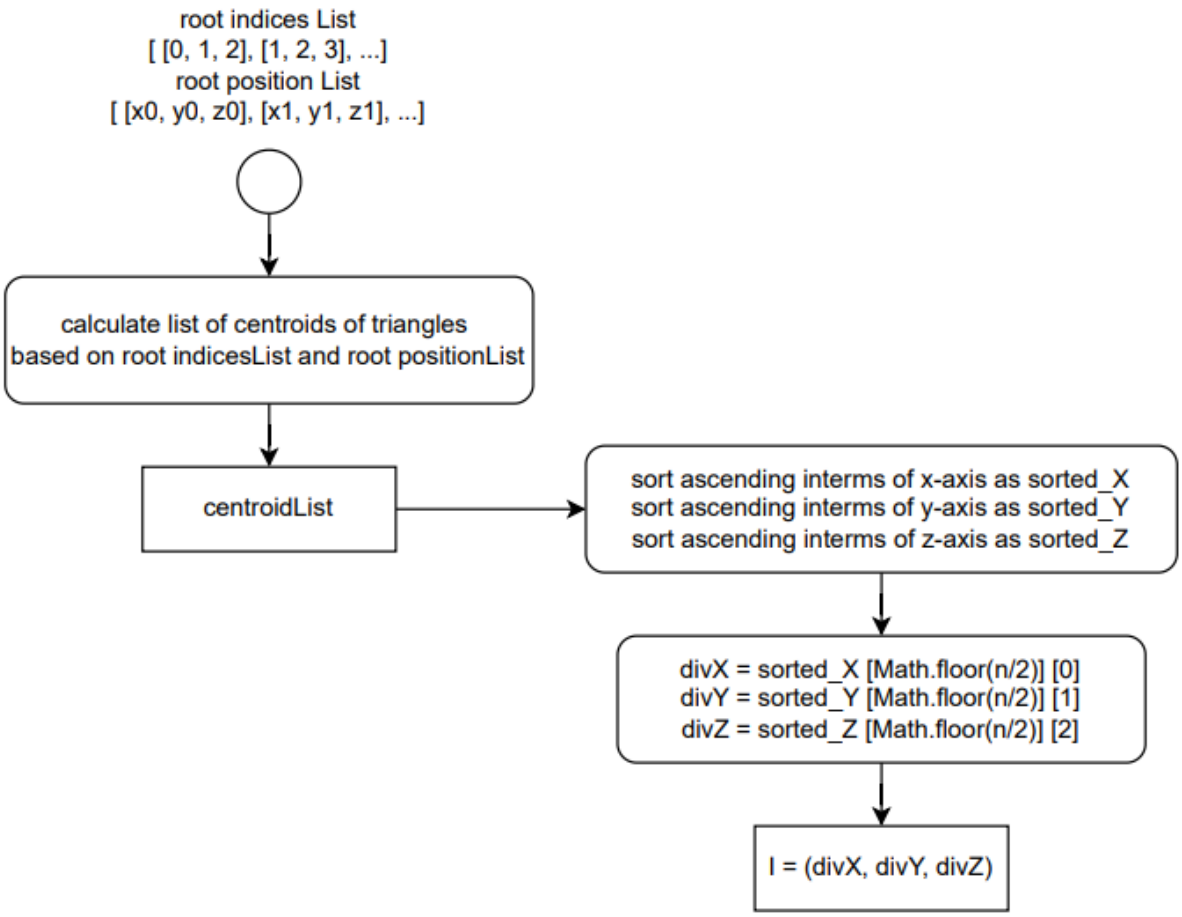

Figure 3.8 Choosing Dividing Point in Adaptive Octree Tiling

We traverse each sub array in the *indicesList* to get the index position in the *positionList*, and based on that the centroid of each triangle will be calculated. The result will be added into an array called *midPointList*. Each centroid will be saved in the format of {x: *value*, y: *value*, z: *value*}.

Next, we proceed to sort the midPointList in ascending order for each of the three coordinates: x, y, and z. After performing three separate sorts, we select the element that situates the middle part of the list, reflecting the desired symmetric properties mentioned earlier.

For the sorted list based on the x-coordinate, we designate the x's value of its middle element as divX. Similarly, for the y-coordinate sorted list, divY is chosen as the y's value of its middle element. Lastly, divZ is set to the z's value of the middle element of the sorted list based on the z-coordinate.

3.3.5 Repeated Partitioning Process

3.3.5.1 Divide into 8 Tiles (Octree)

Our methods are to dividing the scene into 8 smaller parts. Each part represents a tile, which in this case is defined as Tile class in Section 3.3.1. However, there will be 8 tiles after each partition, so we have to declared 8 indices lists, used for storing the indices information of each tile. The name of each tile will base on the *xyz* coordinates that are shown in Figure 3.6.

```
let indiceList_000 = [];
let indiceList_001 = [];
let indiceList_010 = [];
let indiceList_011 = [];
let indiceList_100 = [];
let indiceList_101 = [];
let indiceList_110 = [];
let indiceList_111 = [];
```

Now, the most important part, we start to assign the triangles to each tile base on their position.

With the *indicesList*, we once again traverse through each element, to calculate the centroid of that triangle.

Let each centroid of a triangle be M(midX, midY, midZ). To define the triangle belongs to which tile, we will compare the coordinates of the mid-point of triangle to the point to divide of bounding volume. The detailed of conditions when comparing and which tile that the triangle belongs is shown in Figure 3.5 (block diagram of this stage). The objective is to determine which of the eight partitioned parts of the root bounding volume contains the triangle.

Below is a table describing the details of bounding volume corresponding to each tile (part) from the root bounding volume. We denote the divVol[x, y, z] is the point to divide and Vol[minX, minY, minZ, maxX, maxY, maxZ] is the root bounding volume.

Table 3.1 Bounding Volume of each Tile based on the Root Bounding and Dividing Point

xyz	minX	minY	minZ	maxX	maxY	maxZ
111	midVol.x	midVol.y	midVol.z	Vol.maxX	Vol.maxY	Vol.maxZ
110	midVol.x	midVol.y	Vol.minZ	Vol.maxX	Vol.maxY	midVol.z

101	midVol.x	Vol.minY	midVol.z	Vol.maxX	midVol.y	Vol.maxZ
100	midVol.x	Vol.minY	Vol.minZ	Vol.maxX	midVol.y	midVol.z
011	Vol.minX	midVol.y	midVol.z	midVol.x	Vol.maxY	Vol.maxZ
010	Vol.minX	midVol.y	Vol.minZ	midVol.x	Vol.maxY	midVol.z
001	Vol.minX	Vol.minY	midVol.z	midVol.x	midVol.y	Vol.maxZ
000	Vol.minX	Vol.minY	Vol.minZ	midVol.x	midVol.y	midVol.z

Once the condition is satisfied, the subarray indices will be added to their corresponding *indicesList_xyz*. This process iterates until all indices (triangles) are assigned to their appropriate tiles.

As mentioned in Section 3.3.1, the *size* represents the number of triangles in that Tile. Hence, the value for *size* of each tile will equal the length of *indicesList_xyz*.

After have a full list of indices and the size of each tile, we have to re-calculate the bounding volume. Each tile will have their bounding volume based on Table 3.1, additionally, these bounding volume would be used as the root bounding volume for the incoming partition of each tile if it is needed.

```
const tile_000 = new Tile(
            level,
            "000",
            indiceList_000,
            0,
            0,
            0,
            indiceList_000.length,
            volume_000,
            uri_000,
        );
```

The code above illustrates the code for define a new Tile based on everything we calculated. The *positionList*, *normalList* or *colorList* are temporarily set as 0, since the tile may have to have a new partition stage, so let those list be 0 would save us the time when dividing the scene. Besides, the *uri_000, "000",* and *level* are additional information that help us to distinguish each tile from others.

We declare 8 tiles with the same code above, then sequentially add each tile into the *TileList* that declared in Section 3.3.2.

```
TileList.push(tile_000);
TileList.push(tile_001);
TileList.push(tile_010);
TileList.push(tile_011);
TileList.push(tile_100);
TileList.push(tile_101);
TileList.push(tile_110);
TileList.push(tile_111);
```

After this stage, we have our first level of tiling and the initial set of tiles. In the coming section, we will explain what to do with the *TileList* for a successful partitioning process.

3.3.5.2 Checking conditions and further subdivisions

The conditions of this process are the *size* of each tile must be smaller or equal to the maximum number of triangles that the system set, and there would be no tile with the size of 0.

We just have our initial *TileList* in the previous step. We now will traverse through each tile in the *TileList* to check the condition. However, we cannot use the traditional loop to traverse since it only iterates through a fixed size of array, and the *TileList* is not fixed.

Remember that any tile in the list could be tiled again as Section 3.3.5.1, and every time the event occurs, there will be another 8 tiles added to the *TileList*. Hence this list's length is dynamic, and we need to find another type of traversal.

```
while (i < TileList.length) {
        if (TileList[i].size > maxTriangle) {
            const tile = TileList.splice(i, 1);
            const level = tile[0].level + 1;
            divideOctree(
                tile[0].indiceList,
                tile[0].boundingVolume,
                tile[0].uri,
                positionList,
                TileList,
                level,
            );
        } else if (TileList[i].size === 0) {
            TileList.splice(i, 1);
        } else {
            i++;
        }
    }
return TileList;
```

We use the *while* loop with the variable *i* that denoted as 0, which is used as an index to access elements, at the beginning for the traversal. This *while* loop runs as long as the variable *i* is less than the length of the *TileList* array.

As you can see from the code, there are 3 main condition that are checked for each tile:

- If the *TileList[i].size* > *maxTriangle*: this check if the number of triangles in the current Tile is greater than the requirement. If this condition is true, it means this tile needs to be subdivided further.
    - If the tile needs to be subdivided, it is removed from the *TileList* array using the *splice* method. The *splice* method returns an array containing the removed elements, and in this case, it removes one element from the *TileList* array and stores it in the *tile* variable
    - The *level* variable is set to the *level* property of the removed tile object plus 1. This indicates that the new subdivided tiles will be at a higher level than the current tile.
    - The *divideOctree*, is the function that explains in Section 3.3.5.1, which is divide the tile into 8 sub-tiles based on the properties of the removed tile object. This function receives various arguments, such as *indiceList*, *boundingVolume*, *uri*, *positionList*, *TileList* and *level*
    - *indiceList*, *boundingVolume*, *uri* and *level* are taken from the removed tile, while the *positionList* we take from the public root list at the beginning. Lastly, the *TileList* is used for the function after division to update the tiles.
    - Note that the index *i* after removing one tile from the list will remain unchanged.
- If the *TileList[i].size* == 0, this means that the tile is empty, and will be removed from the list by using the *splice* function. And the index *i* in this condition still remains the same.
- If none of the previous conditions are met, it means the tile is neither too large nor empty, so we can proceed to the next tile. In this case, we simply increment the value of *i* by 1, moving to the next index in the *TileList* array.

The *while* loop will continue iterating through the *TileList* array until the *i* has reached the end of the array, and the system will return the final *TileList* version.

3.3. Tileset Complication

Now we have finished the Partitioning process, but to stream the scene onto the application, we need to convert the whole *TileList* into a JSON tileset (as described in Chapter 1). The figure below illustrates the process of creating a JSON tileset.

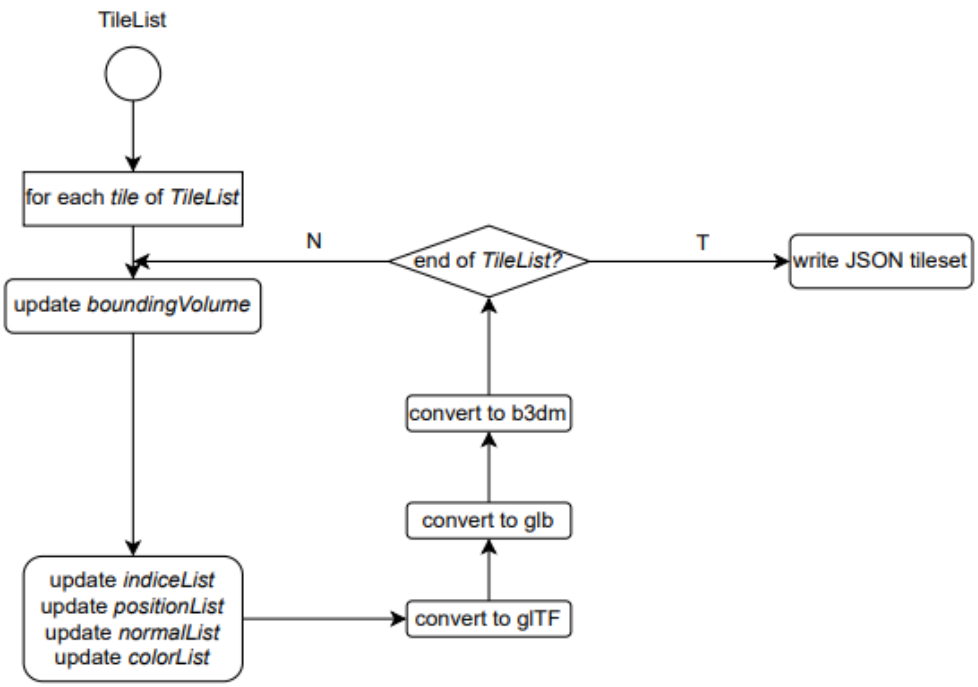

Figure 3.9 The block Diagram Of Tileset Complication Stage

Each tile in the *TileList* is a children of the root node in the tileset. Each children will have their own bounding volume, geometricError and content. The content of each children is the *indiceList, positionList, normalList and colorList* which are stored in each tile. The contents are saved in the B3DM format, so it would take a few steps to make a full tileset.

3.4.1. Updating Full Information of Each Tile

Through Section 3.2, the system has returned a list of *TileList*. However, each tile in the list only contains the value of original *indiceList*, which has the index corresponding to the position of original *positionList, normalList, colorList.* Since each tile has its own set of lists, there will be needed to re-index in each list.

Additionally, the bounding volume that is saved in each tile is the volume when dividing, not the final version of it. When we partition the data, each triangle is defined into a tile based on their centroid position coordinate, not their vertex coordinate, so there will be some occasions where a big triangle stays in both Tile 1 and Tile 2 when dividing. However, that triangle is assigned in Tile 1 since the centroid is located inside that tile. 3D Tiles or tileset defines the bounding volume must contains its full renderable content, so if we continue with the saved volume in the list, some part will be not rendered.

3.4.1.1 Updating boundingVolume of Each Tile

The method of this step is the same as calculating the root bounding volume that we mentioned in Section 3.2.2.7. The value of each sub array in the *indiceList* is the index of the corresponding indices position in the total *positionList*.

We again define a new *boundingVolume* variable the *minX*, *minY*, *minZ* equal to Infinity and *maxX*, *maxY*, *maxZ* to negative Infinity. Traversing through every *indice* of the *indicesList* to update the boundingVolume based on the position of each *indice*.

```
for (const indices of indiceList) {
    for (const index of indices) {
        let position = positionList[index];
         minX = Math.min(minX, position[0]);
         minY = Math.min(minY, position[1]);
         minZ = Math.min(minZ, position[2]);
         maxX = Math.max(maxX, position[0]);
         maxY = Math.max(maxY, position[1]);
         maxZ = Math.max(maxZ, position[2]);
     }
 }
```

Finally, we can update the final *boundingVolume* of each tile.

```
const boundingVolume = {
    minX: minX,
    minY: minY,
    minZ: minZ,
    maxX: maxX,
    maxY: maxY,
    maxZ: maxZ,
};
return boundingVolume;
```

3.4.1.2 Updating indiceList, positionList, normalList, colorList of Each Tile

For further understanding of this step, we propose an example:

We have the *indiceList* that is being stored in tiles like this: [[0,1,2], [1,2,4], [2,4,6]]. As you can see, the value varies from 0 to 6, which means corresponding to the first to the seventh position in original *positionList.*

When we create a tile, to reduce the size of a tile, we need to simplify the lists. From the array above, we can re-index as [[0,1,2], [1,2,3], [2,3,4]]. The value “3” now represents the “4” in the old list, which means we have removed the redundant element out of the fours list.

We develop an asynchronous function that takes five parameters: a tile which are being updated, *indiceList* of the tile, *positionList*, *normalList*, *colorList* of the original scene. The purpose of this function is to simplify the given octree data by removing redundant vertices and updating the associated index lists accordingly.

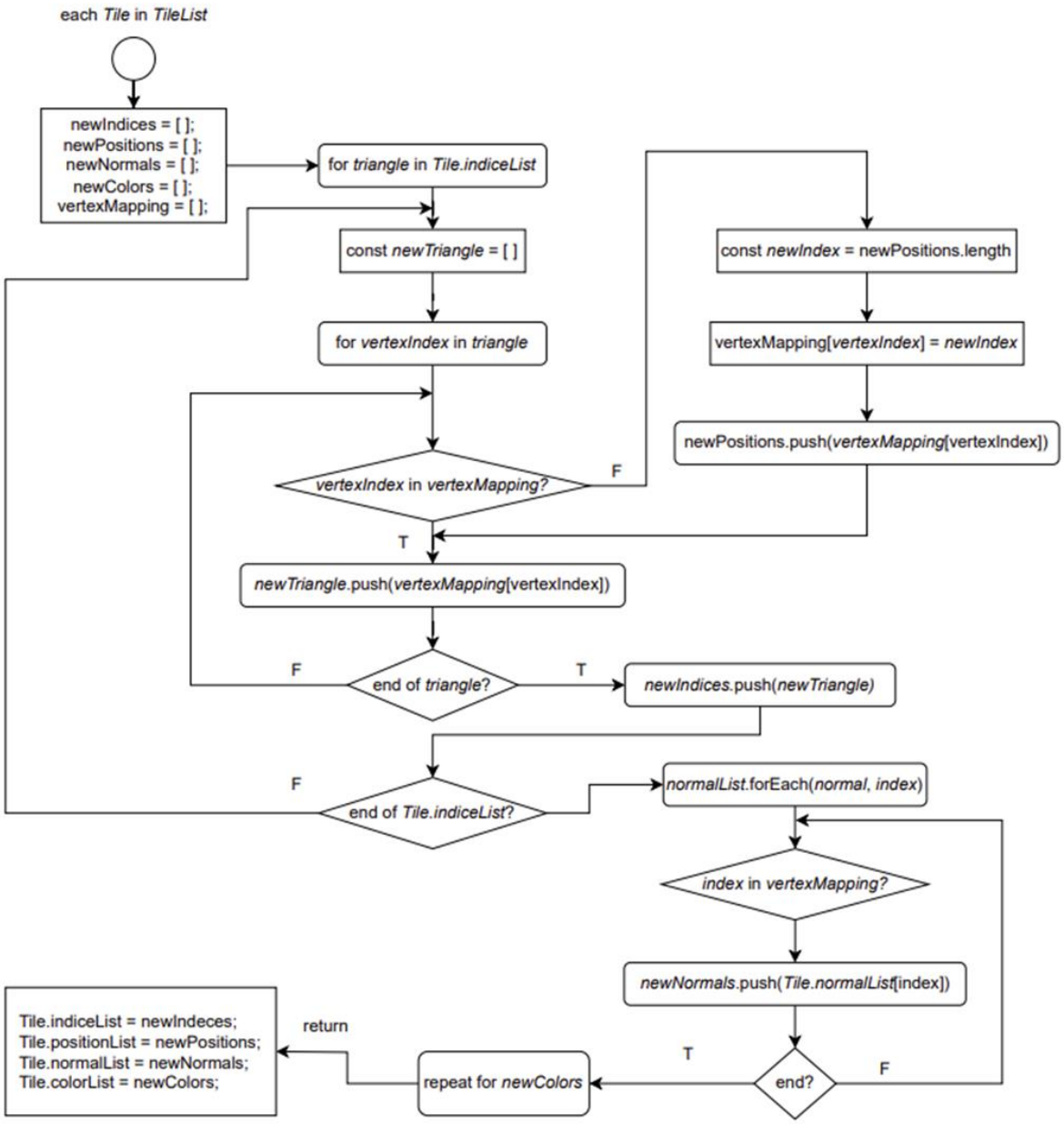

Figure 3.10 Update Attributes for each Tile

The function declares four new arrays: *newIndices*, *newPositions*, *newNormals*, and *newColors*, which will be used to store the simplified data. We also create a *vertexMapping* object which is used to keep track of unique vertices and their corresponding indices in the *newPositions* array.

```
indiceList.forEach((triangle) => {
    const newTriangle = [];
    triangle.forEach((vertexIndex) => {
        if (!(vertexIndex in vertexMapping)) {
            const newIndex = newPositions.length;
            vertexMapping[vertexIndex] = newIndex;
            newPositions.push(positionList[vertexIndex]);
        }
        newTriangle.push(vertexMapping[vertexIndex]);
    });
```

```
    newIndices.push(newTriangle);
});
```

The function starts iterating over each triangle in the *indiceList*. For each vertex index in the triangle, it checks if the vertex index exists in the *vertexMapping* object. If it is a new vertex, it adds it to the *newPositions* array and updates the *vertexMapping* to store the old vertex index as the key and the new index in the *newPositions* array as the value. The *newTriangle* array is updated with the new index for each vertex, resulting in a simplified triangle.

```
normalList.forEach((normal, index) => {
    if (index in vertexMapping) {
        newNormals.push(normalList[index]);
    }
});
colorList.forEach((color, index) => {
    if (index in vertexMapping) {
        newColors.push(colorList[index]);
    }
});
```

After updating the indices and positions, the function iterates over the *normalList*. For each normal vector, it checks if the corresponding index is present in the *vertexMapping*. If the vertex index exists in the mapping, it means that the vertex is part of the simplified data, and the normal vector is added to the *newNormals* array accordingly.

Similar to the previous step, the function iterates over the *colorList*. For each color, it checks if the corresponding index is present in the *vertexMapping*. If the vertex index exists in the mapping, it means that the vertex is part of the simplified data, and the color is added to the *newColors* array accordingly.

```
Tile.indiceList = newIndices;
Tile.positionList = newPositions;
Tile.normalList = newNormals;
Tile.colorList = newColors;
```

Finally, after processing all triangles, normals, and colors, the function updates the *Tile* object by assigning the *newIndices*, *newPositions*, *newNormals* and *newColors* array to the respective properties of the *Tile* object.

3.4.2. Converting a Tile to B3DM Format

3.4.2.1 Converting a Tile to glTF Format

After successfully updating the full list of tiles on the previous steps, we are now creating a function that takes each *Tile* object as input and generates a glTF file based on the data stored in the *Tile*. The glTF file is a 3D model format that represents 3D scenes and objects, which can be rendered by various 3D engines and applications.

```
const glTFData = {
    asset: { version: "2.0", generator: "COLLADA2GLTF" },
    accessors: [],
    buffers: [],
    bufferViews: [],
    meshes: [],
    nodes: [{ children: [1] }, { mesh: 0 }],
    scenes: [{ nodes: [0] }],
    scene: 0,
};
```

The function starts by creating an empty glTF data object named *gltfData*. This object will hold all the necessary information to describe the 3D model. As you can see the object has all the properties that are needed in a glTF that have been mentioned in Chapter 1.

```
let indiceList = Tile.indiceList.flat();
let positionList = Tile.positionList.flat();
let normalList = Tile.normalList.flat();
let colorList = Tile.colorList.flat();
```

Next we flatten all the arrays, converting them to 1D arrays since they are stored in 2D arrays. Flattening is necessary because glTF uses 1D buffers for data storage.

```
//Encode indices
const indicesBuffer = new Uint16Array(indiceList);
const indicesDataURI = encodeToDataURI(indicesBuffer.buffer);
//Create bufferView indices
const indicesBufferView = createBufferView(indicesBuffer, 0);
//Create accessor indices
const indicesAccessor = createIndicesAccessor(
    indicesBuffer,
    0,
    "SCALAR",
    5123,
    max,
);
```

For each list, we encode the flattened arrays into data base64. Then creating buffer views and accessors for each of the attribute data (indices, positions, normals, colors). Buffer views define the binary data range within the buffers, and accessors define how to interpret that data (e.g., data type, component type, etc.). The detailed of the sub function will be shown in the Appendix.

After encoding, creating buffer views and accessors for each attribute, we define mesh and primitives. The function creates a mesh with a single primitive. Primitive defines the rendering mode (triangles), the indices buffer, and the attributes (positions, normals, colors) with their corresponding index of accessors.

```
const primitive = {
    mode: 4,
    indices: 0,
    attributes: {
        POSITION: 1,
        NORMAL: 2,
        COLOR_0: 3,
    },
```

```
};
const mesh = {
    primitives: [primitive],
};
```

In the next step, we add respectively the indices, positions, normals and color accessor to the *accessors* property. For the *meshes* property, we add the *mesh* that declared above. Continuing the same for the *buffers* and *bufferViews* properties.

Finally, we convert the *glTFData* object to a JSON string representation. The function writes the generated JSON data to a glTF file with the name based on Tile.uri and it is saved in the specified directory.

```
const gltf = JSON.stringify(glTFData, null, 2);
await fs.promises.writeFile(
    "./middleware/output/gltf/" + Tile.uri + ".gltf",
    gltf,
    "utf8",
    (err) => {
       if (err) {
           console.error("Error writing glTF file:", err);
       } else {
            console.log("glTF file written successfully!");
       }
   },
);
```

In summary, this stage takes each *Tile* object in the *TileList*, converts it into glTF format, and saves it as a glTF file.

3.4.2.2 Converting a glTF File to GLB Format

In order to convert to b3dm file format, we need to have a middle step is to convert to glb format.

```
async function convertGLTFtoGLB(Tile) {
    // Run the conversion
    const inputFile = __dirname + Tile.uri + ".gltf";
    const outputFile = "./middleware/output/glb/" + Tile.uri + ".glb";
    try {
        const command = `gltf-pipeline -f -i ${inputFile} -o
${outputFile}`;
        const result = await runCommand(command);
        console.log(result);
    } catch (error) {
        console.error(`An error occurred: ${error.message}`);
    }
}
```

The function is used for automating using terminal to convert the glTF file to glb file with the input is the path to the glTF file based on the *Tile.uri*, and the specific file. The process uses the *gltf-pipeline* as mentioned in Section 1.3.4.

3.4.2.3 Converting a GLB File to B3DM Format

Finally, with the same method of converting glb file to b3dm file mentioned in Section 3.4.2.2 and section 1.4, we have successfully converted all the tiles into the appropriate format that contains data of each tile.

3.4.3. Write JSON Tileset

In this step, we create a tileset.json file contains all the information that we have calculated and converted. Based on the 3D tiles specifications, we first declare an object *tileset.*

```
let tileset = {
        asset: {
            version: "1.0",
        },
        geometricError: geometricError,
        root: {
            refine: "ADD",
            transform: positionMatrix,
            boundingVolume: {
                box: rootBox,
            },
            geometricError: geometricError,
            children: [
                {
                    geometricError: geometricError,
                    transform: translationMatrix,
                    boundingVolume: {
                        box: rootBox,
                    },
                    children: [],
                },
            ],
        },
    };
```

The object contains some main properties: *asset*, *geometricError*, and *root.* The *asset* contains the *version* of the 3D Tiles specification. The *geometricError* is a parameter use to control the level of detail of individual tiles within a tileset. The *geometricError* helps manage the trade-off between visual quality and performance in rendering 3D data. As explained in Chapter 1, setting the *geometricError* involves finding a balance between performance and visual quality. If the *geometricError* is too high, the 3D scene may look blocky and lack fine details, but the rendering performance will be better. Conversely, if the *geometricError* is too low, the scene will be highly detailed but may lead to performance issues, especially for large datasets. In this process, we choose the *geometricError* equals to 100 which is usually used in general computation. Noted that every level of details or tile must have its *geometricError.*

The *root* represents the root of the scene, and there is *refine* property which is the method of refinement. We set *refine* as “ADD”, meaning when the viewpoint is reached to a level of detail, then a new tile will be added to the current scene.

The *boundingVolume* we set in the root is the bounding of the whole scene, so it equals to the original root bounding volume that we calculated in Section 3.2.2.7.

Moreover, in the root, we have a *transform* property. This *transform* is a matrix we use to set the position of the scene in the world coordinates. We use the library developed by Cesium JS to computing the transformation to the world coordinates based on the longitude, latitude and the height.

```
"transform": [
  -0.9659258262890682,
  -0.25881904510252085,
  0,
  0,
  0.09695531991670876,
  -0.36184217999325163,
  0.9271838559409777,
  0,
  -0.2399728402291171,
  0.8955908321716735,
  0.37460659001467134,
  0,
  -1531296.6870677804,
  5714877.037598854,
  2374409.1492368816,
  1
],
//example of an transform matrix
```

The matrix is calculated by using the *headingPitchRollToFixedFrame* function, and then convert to a 2D array 4x4. We set this matrix in the root for the full transformation of the scene.

In the root, we have the first level of children. In this *root.children* , besides the *geometricError* and *boundingVolume* with the same value to the *root*, there is a new *transform* property. However, this *transform* is different from the one in *root since* it represents the translation of y-coordinate up to the z-coordinate up. The reason for having this kind of transformation is explained in Section 1.1.6.

To computing this matrix, we have to take the center point of the whole scene and then using the *fromTranslation* function that developed by the Cesium.

Finally, in *root.children* will have *children* property, which is an array, contain the list of tiles that we implement above. These children will be the second level of the hierarchy, and for easy understanding, we set all tiles in this level.

```
for (const Tile of TileList) {
    const newBox =
                calculate.calculateBoxTilesetYupZup(Tile.boundingVolume);
    const newUri = "./middleware/output/b3dm/" + Tile.uri + ".b3dm";
```

```
    tileset.root.children[0].children.push({
        boundingVolume: {
            box: newBox,
        },
        content: {
            uri: newUri,
        },
        geometricError: 0,
    });
}
```

We iterate through the *TileList*, and each *Tile* represents a *children* element in the *root.children.children*. For each *Tile*, there will be *geometricError*, *boundingVolume*, and the *content*.

The *geometricError* we set all of them be 0, while the *content.uri* will be the path that stores the b3dm file of that *Tile*, based on the *Tile.uri* property that we saved for each tile in previous steps. The *boundingVolume* represents the boundary volume of that tile, however, we have to recalculate the *boundingVolume* since the last version of it, we store the bounding volume where the original coordinate (the center point of the scene) was (0,0,0). Now, after converting the scene into the earth coordinate with the longitude, latitude and height, there will be needed to move the bounding volume a few meters higher. Moreover, since now the y-coordinates up has turned to z-coordinates up, we also need to modify the bounding volume to fit that transformation.

After having all the properties calculated, we add them to the *root.children.children* array as an object. And finally, we can convert the whole tileset into the JSON file using the *fs.writeFileSynce* function.


```
"children": [
  {
    "boundingVolume": {
      "box": [
        -593.4553527832031,
        -437.20672607421875,
        0.0010000000474974513,
        143.46365356445312,
        0,
        0,
        0,
        270.14141845703125,
        0,
        0,
        0,
        0
      ]
    },
    "content": {
      "uri": "./middleware/output/b3dm/__0_000__1_010_.b3dm"
    },
    "geometricError": 0
  },
```


Figure 3.11 Example of a children tile in the JSON tileset

CHAPTER 4. IMPLEMENTATION AND EVALUATION

In the previous chapter, we have shown how the partitioning methods were created and, in this chapter, we will apply the tileset into a real project web application, hence we can have a deeper comparison between the two methods and how it satisfies the thesis targets.

4.1. Implementation on CesiumJS

CesiumJS is an open-source JavaScript library for creating world-class 3D globes and maps with the best possible performance, precision, visual quality, and ease of use. Developers across industries, from aerospace to smart cities to drones, use CesiumJS to create interactive web apps for sharing dynamic geospatial data.

We use it as the main source to render and comparing the tilesets. The CesiumJS could be downloaded as a package in NodeJS in the name “cesium”.

The codes below show how the server can implement the tileset to the website:

```
// function definition
window.startup = async function (Cesium) {
  "use strict";
  // code will be shown after
};
```

The *window.startup* function is defined as an asynchronous function, which means it can use the *await* keyword to handle asynchronous operations. The *“use strict”* directive enables strict mode in JavaScript, which enforces stricter syntax rules and prevents the use of undeclared variables, among other benefits.

```
//Creating the Viewer
const viewer = new Cesium.Viewer("cesiumContainer", {
  baseLayer: Cesium.ImageryLayer.fromProviderAsync(
    Cesium.TileMapServiceImageryProvider.fromUrl(

Cesium.buildModuleUrl("/cesium/Source/Assets/Textures/NaturalEarthII"),
    ),
  ),
  geocoder: false,
});
```

This code initializes a Cesium Viewer, which is responsible for managing the rendering, camera, and various other aspects of the scene. Cesium Viewer provides the user interface for displaying the 3D globe or map and interacting with the scene. It creates a viewer within an HTML element with the ID *“cesiumContainer”*. The viewer is configured with a base layer, which is a *TileMapServiceProvider* using the dataset of *NaturalEarthII*.

```
//Set tileset
const osmTileset = await Cesium.Cesium3DTileset.fromUrl(
  "../tileset.json",
```

```
  { enableDebugWireframe: true },
);
viewer.scene.primitives.add(osmTileset);
osmTileset.maximumScreenSpaceError = 100;
viewer.zoomTo(osmTileset);
```

This part of the code loads a 3D tileset representing buildings from a *tileset.json* file located in the parent directory ("*../tileset.json"*). The *Cesium3DTileset.fromUrl* method asynchronously loads the tileset. Once loaded, it adds the tileset to the viewer's scene primitives.

The *maximumScreenSpaceError* property is set to control the level of detail for the tileset based on its distance from the camera. This property allows you to manage the visual quality and performance of rendering 3D tilesets, especially when dealing with large datasets. The *maximumScreenSpaceError* property allows you to specify the maximum allowable screen space error for a 3D tileset. When the camera's view frustrum intersects with a tile (i.e., the tile is visible on the screen), CesiumJS calculates the screen space error for that tile. If the calculated screen space error exceeds the value of *maximumScreenSpaceError*, CesiumJS will automatically switch to a more detailed level of the 3D tile to reduce the error and improve visual quality. Conversely, if the calculated error is smaller than *maximumScreenSpaceError*, CesiumJS may switch to a lower level of detail to improve rendering performance.

Finally, the viewer is zoomed to fit the loaded 3D tileset. The *viewer.zoomTo* function is used to automatically adjust the camera position and field of view to fit the entire extent of the specified *osmTileset*.

4.2. Demo Application

4.2.1 Application Explanation

The Block Diagrams of this application are based on Figure 2.2 and Figure 2.3. Actor will have 3 main activities and all the information after the Partitioning process will be stored at the server.

To begin the process, the user initiates a request to the server, prompting a transition to the Welcome Page. This initial page serves as a platform for users to input their requirements. It includes sections for uploading the glTF file, specifying the maximum triangle count, and selecting a tiling method—either Static Tiling or Adaptive Tiling.

Once the user submits the request, the server receives the uploaded file along with the associated tiling parameters. The server then employs the chosen tiling approach based on the user's preference. Upon completion of the tiling process, the server promptly sends back a status update to the client. Subsequently, the user is seamlessly directed to the next stage—the Rendering Page.

The Rendering Page The Rendering Page is built on CesiumJS, as elaborated in Section 4.1. This page is responsible for the visual rendering of the generated tileset. Following this step, the subsequent content will encompass the outcomes and visual representations of each page.

The Rendering Page will base on the position of the viewer (latitude, longitude, height, rotating,..) to request the server the information of suitable tiles under that view. These information will be returned by server in terms of b3dm files, which we have extracted and stored in Section 3.4.2. Actor can zoom in, zoom out, span or rotate the scene, and the Cesium JS would automatically send request to server is the position of viewer after zoom/span/rotate has never loaded before. Moreover, Actor can see the information of the streaming tiles (boundingVolume,..) by using Inspector.

4.2.2 Result

Figure 4.1 Welcome Page

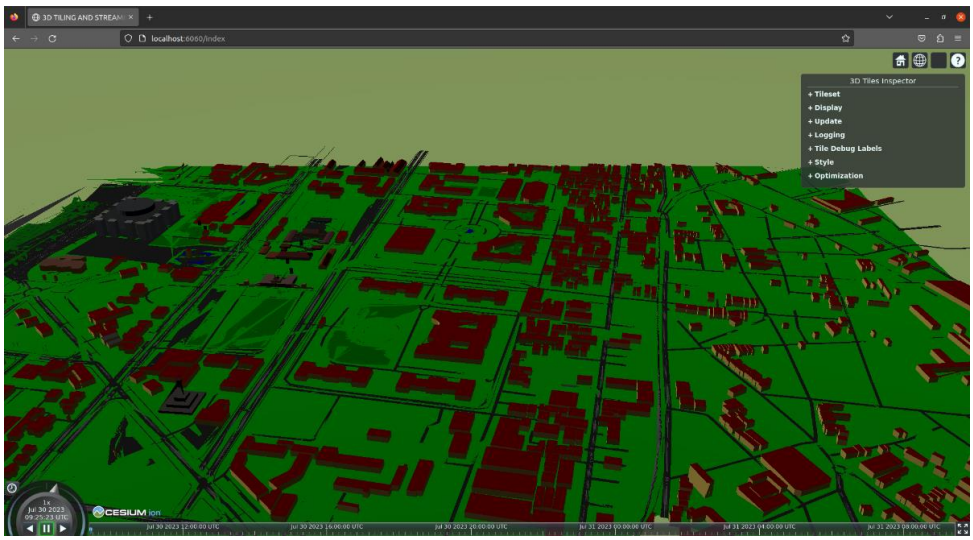

Figure 4.2 Rendering Page (Cesium JS)

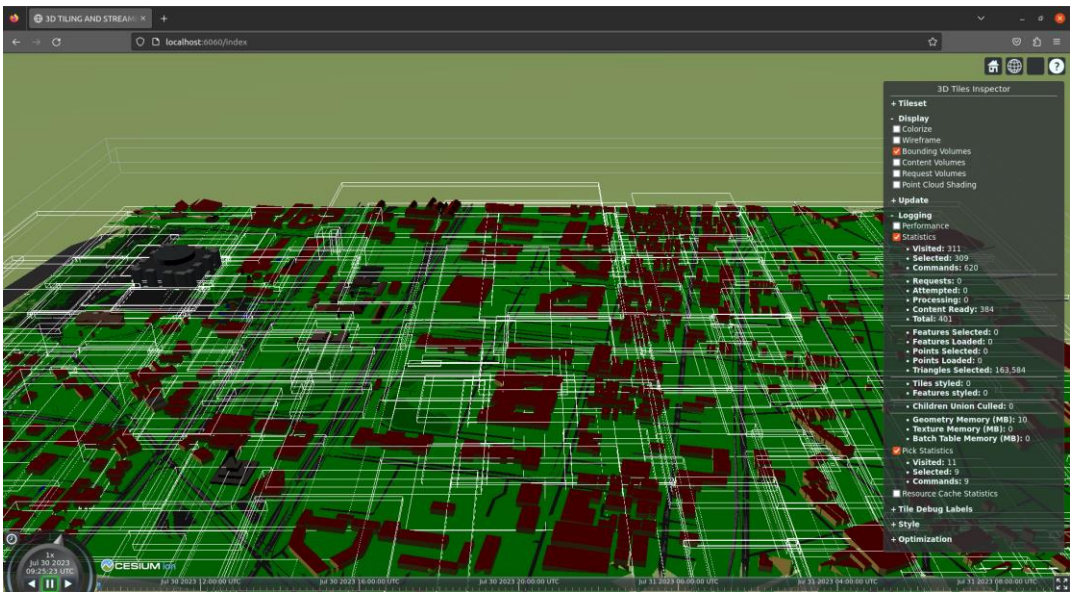

Figure 4.3 Rendering Page with detailed Bounding Volume with statistics

Above are some figures that illustrate the full implementation of 3D Tiles with our method of partitioning. In the next section, we will drive into the statistics of each method and make some comparisons to determine if the targets are met.

4.3. Measurement

The first objective, which is to partition and transform extensive glTF files into JSON tileset to be streamed and visualize on Cesium, is successfully finished. The next objective we are concerning about is the speed of rendering tileset of each method.

We have two methods: Static Tiling and Adaptive Tiling. For the most objective measurements, we choose the same file glTF with the size of 61.8 MB as the input of both partitioning methods. This size of file would be large enough to see the difference in the measurements.

We divide the measurement into many cases. First, we compare the rendering speed of both partitioning methods while they have the same maximum number of triangles per tile. Next, we may compare the rendering's speed when varying the value of maximum number in each method. Then we can conclude the results if they reach the targets that we set above.

4.3.1. Same Maximum Number Of Triangles Per Tile

We take the 61.8 MB glTF file above and put it into the tiling process for both methods and get the tilesets. To calculate the rendering's speed of each method, we add an event called *initialTilesLoaded* into the server. This event will return to the terminal the duration time when the first tiles are loaded based on the initial viewer. Additionally, during the rendering process, we add an *addProgress* event to detect

real-time the number of tiles which are being rendered. These numbers of processing tiles will be stored into an array and then will be drawn into a line graph for the comparison.

4.3.1.1 2000 Triangles Per Tile

The two figures below illustrate the time for rendering from two methods based on the same source model with the maximum number of triangles per tile is 2000.

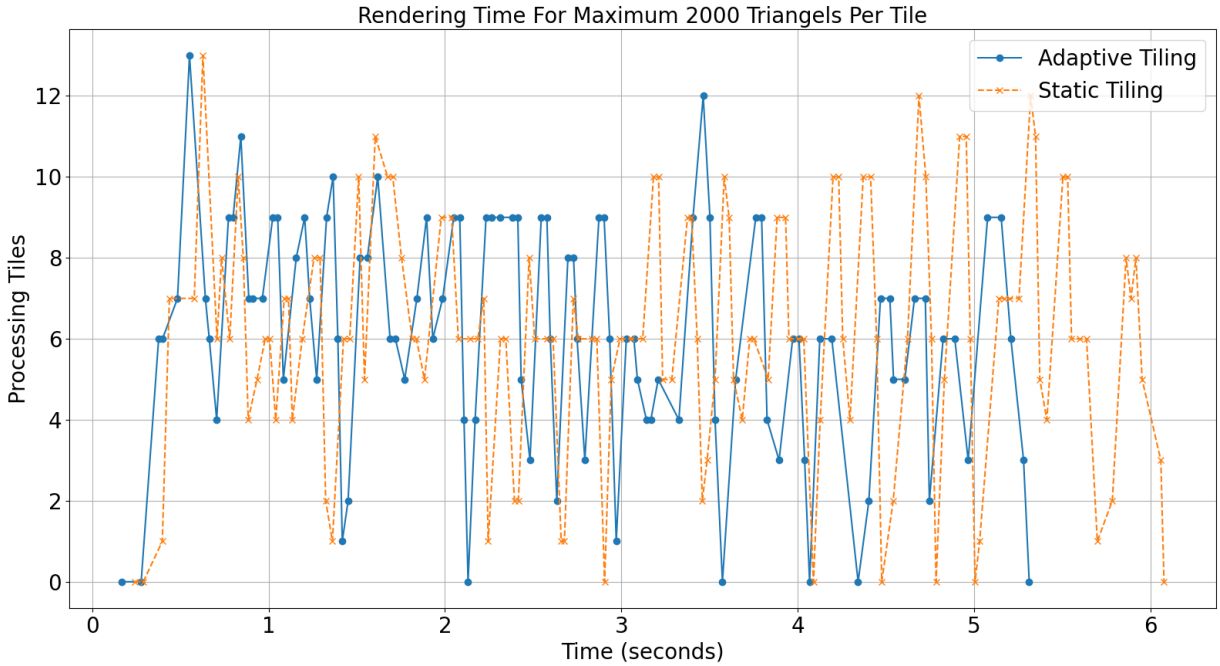

Figure 4.4 Rendering Time For Maximum 2000 Triangles Per Tile

In general, the Adaptive Tiling method exhibits superior rendering speed compared to the Static Tiling approach. The tileset generated using the static octree method takes approximately 6.076 seconds to render. On the other hand, the tileset created using the adaptive octree method achieves a faster rendering time of 5.312 seconds, surpassing the static method by more than half a second, and nearly 20% faster.

One reason for this shorter time of rendering is the number of tiles. There are fewer tiles loaded, leading to fewer draw calls for a comparable visual result in much less time.

The point to divide in each method is the key point to this development. The adaptive method has a better select the subsection of the model it needs and has a better distribution of triangles in each tile.

Figure 4.5 Result after rendering with Static Tiling (left) and Adaptive Tiling (right)

Figure 4.6 The Statistics with Static Tiling (left) and Adaptive Tiling (right)

From Figure 4.6 above, we can see that Adaptive Tiling has a smaller number of tiles compared to Static Tiling. The Adaptive render selects only 265 tiles while the Static selects 318. However, even the Adaptive has a smaller number of tiles, but it covers a much better range of triangles with the triangles selected at 168,298. This number of Static Tiling has only 164,375.

The results unequivocally highlight the superiority of the Adaptive Tiling technique in achieving a more efficient rendering rate. This approach employs a smaller number of tiles while simultaneously delivering higher levels of detail. The ability to maintain both a reduced tile count and enhanced detail makes Adaptive Tiling the clear winner in optimizing the rendering process.

4.3.1.2 3000 Triangles Per Tile

For more convinces, we change the maximum number of triangles per tile up to 3000. Now we do the comparison the same with previous part.

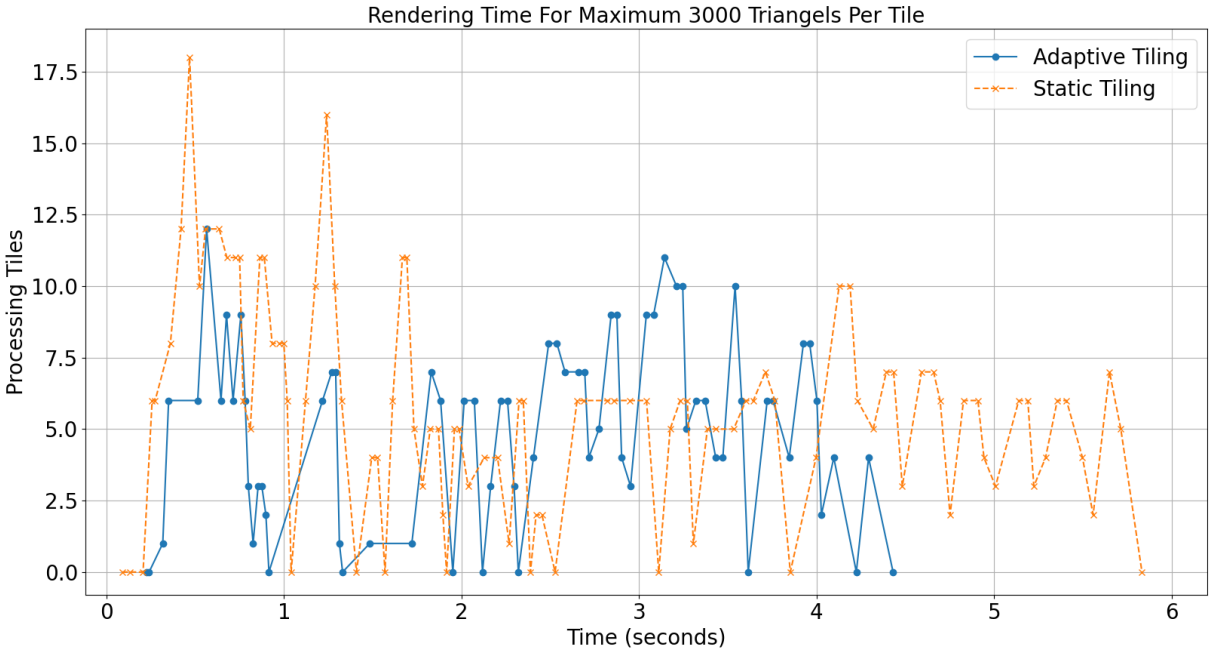

Figure 4.7 Rendering Time For Maximum 3000 Triangles Per Tile

In this scenario, the rendering time for Adaptive Tiling significantly outperforms that of Static Tiling. Specifically, the Adaptive approach completes the rendering process in approximately 4.431 seconds, while the Static Tiling takes around 5.833 seconds (approximately 30% more than the rate of Adaptive). This substantial time difference reinforces our confidence in achieving the project's objectives successfully.

Once again, the figures below vividly demonstrate the crucial significance of the dividing point in each method concerning the efficiency of the tileset. It becomes evident that the optimal selection of the dividing point plays a pivotal role in making the tileset considerably more efficient for both techniques.

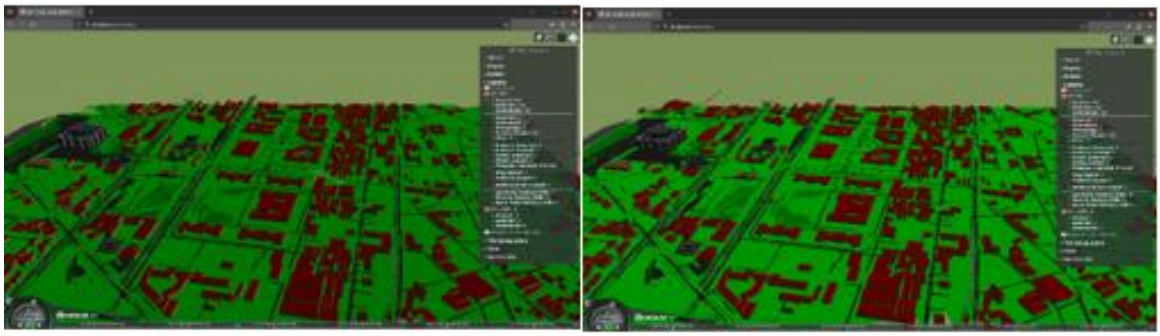

Figure 4.8 Result after rendering with Static Tiling (left) and Adaptive Tiling (right)

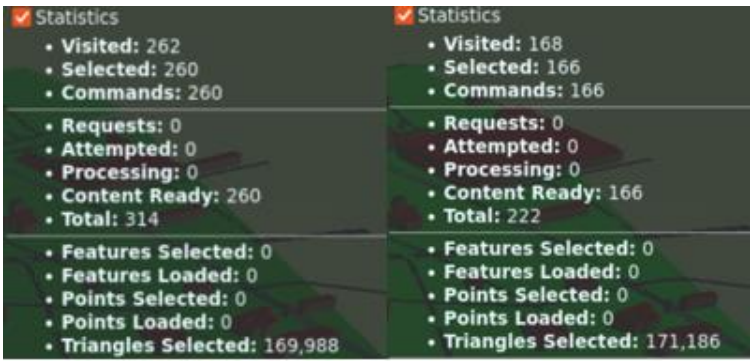

Figure 4.9 The Statistics with Static Tiling (left) and Adaptive Tiling (right)

Once again, the comparison between Adaptive Tiling and Static Tiling reveals that the former employs a smaller number of tiles, specifically 166 tiles, whereas the latter utilizes 260 tiles. Despite this reduction in the number of tiles, the Adaptive approach covers a significantly wider range of triangles, with 171,186 triangles selected, while Static Tiling only covers 169,988 triangles.

This observation emphasizes that Adaptive Tiling achieves a more efficient rendering rate, even with a reduced tile count, due to its ability to encompass a broader range of triangles.

4.3.2. Measurement Conclusions

Through the measurements above, we can tell that Adaptive Octree Tiling does create a tileset with better rendering rate than the Static Octree Tiling based on the time consuming. This measurement has again proved the initial targets have been accomplished.

CONCLUSION

General conclusion

Throughout this thesis, we have introduced and compared two primary methods of partitioning: Static Octree Tiling and Adaptive Octree Tiling. These methods rely on the maximum number of triangles per tile as a critical factor in their tiling process. The resulting tilesets have been meticulously verified and successfully streamed using the Cesium platform.

Among the achievements in this project, the Adaptive method stands out as particularly successful, meeting the targets we initially set. Notably, rendering tilesets generated by the Adaptive method proves to be significantly faster, with speed improvements ranging from 20% to 30% compared to tilesets created using the Static method. This considerable performance gain showcases the efficiency and practicality of the Adaptive approach in real-world scenarios.

However, there are many scenarios of measurement that we can't do due to the limitation of the material, and this rendering speeds in compared to the community's results are still has a lot of work to do.

Future works

One of several things that we can focus on and develop is the Optimization Strategies, which is investigate and develop additional optimization techniques to further improve the rendering performance of both the Static and Adaptive Tiling methods. Exploring advanced algorithms or parallelization approaches may lead to even greater efficiency gains.

Another idea is to have a Dynamic Thresholding, which explores the concept of dynamically adjusting the maximum number of triangles per tile based on the complexity of the 3D scene or the rendering hardware. By dynamically adapting the threshold, the tiling process can optimize tileset generation and rendering for different contexts, leading to more adaptive and responsive visualizations.

REFERENCES

[1] Google Maps Platform documentation [Online].
Available: https://developers.google.com/maps/documentation.

[2] Cesium, *3D Tiles Essentials* [Online].
Available: https://www.cesium.com/why-cesium/3d-tiles/3d-tiles-essentials/.

[3] Cesium, *Introducing 3D Tiles* [Online].
Available: https://www.cesium.com/blog/2015/08/10/introducing-3d-tiles/.

[4] Cesium, "3D Tiles Specification 1.0", submitted to the Open Geospatial Consortium and approved as an OGC Community Standard, December 14, 2018. Available: https://github.com/CesiumGS/3d-tiles/blob/main/3d-tiles-reference-card.pdf.

[5] Epsg.io, *WGS 84 – EPSG:4978.* Available: https://epsg.io/4978.

[6] Cesium, "3D-Tiles Specification" [Online] .
Available: https://github.com/CesiumGS/3d-tiles/tree/main/specification.

[7] The Khronos Group, "glTF 2.0 Specification" [Online].
Available: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html.

[8] The Khronos Group, "GLB File Format Specification" [Online].
Available: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html#glb-file-format-specification.

[9] Cesium, "Batched3DModel" [Online]. Available: https://github.com/CesiumGS/3d-tiles/blob/main/specification/TileFormats/Batched3DModel/README.adoc.

[10] Cesium, "ImplicitTiling" [Online]. Available: https://github.com/CesiumGS/3d-tiles/tree/main/specification/ImplicitTiling.

[11] OpenStreetMap [Online]. Available: https://www.openstreetmap.org/.

[12] OSM2World [Online]. Available: https://www.osm2world.org/.

APPENDIX

The full source-code of this project could be found at:

Github: https://github.com/vtmhieu/Map_Cesium